CS 342:
Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Image by kirkh.deviantart..com. Credit for the slides goes to many people including James Hays, Danny Alexander, Aaron Bobick, and many others. The instructor doesn't claim credit for any slides.

Shanti Mantra

Lyrics in Sanskrit	Meaning in English
ॐ सह नाववतु।	Om, May we all be protected
सह नौ भुनक्तु।	May we all be nourished
सह वीर्यं करवावहै।	May we work together with great energy
तेजस्वि नावधीतमस्तु	May our intelect be sharpened (may our study be effective)
मा विद्विषावहै।	Let there be no Animosity amongst us
ॐ शान्तिः शान्तिः शान्तिः ॥	Om, peace (in me), peace (in nature), peace (in divine forces)

WHAT IS AN IMAGE?

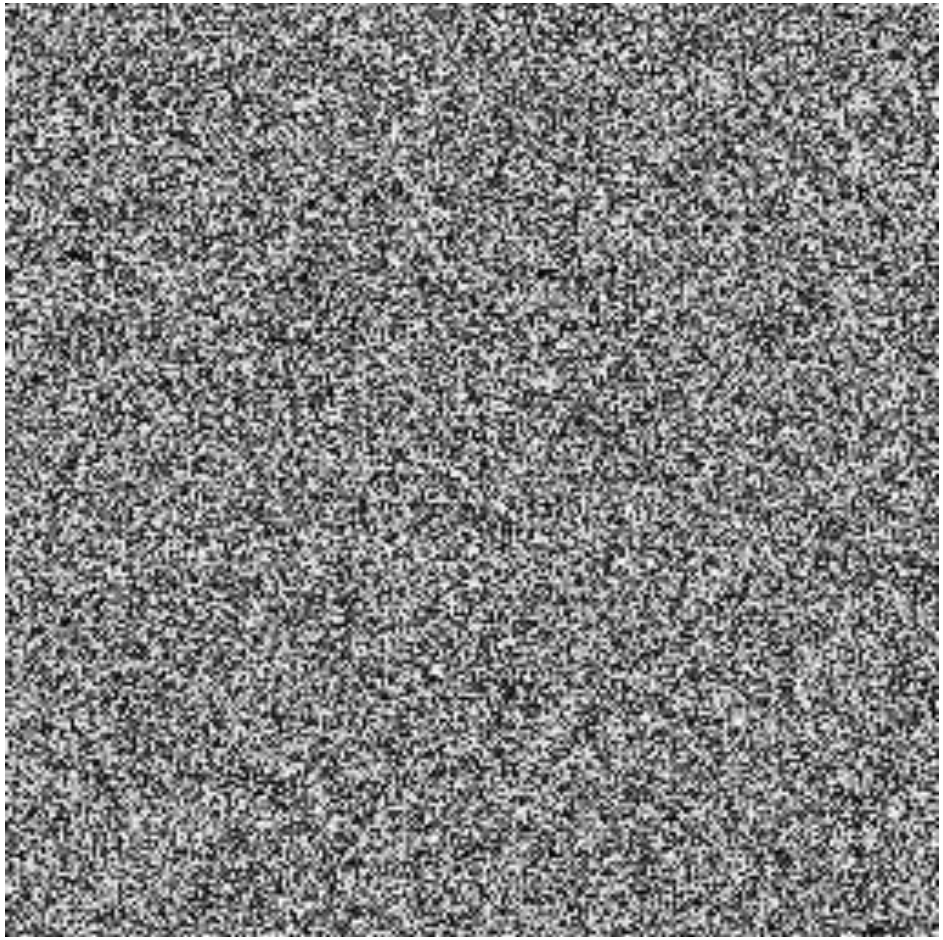
```
>>>  import numpy as np
>>> I = np.random.rand(256,256);
```

Think-Pair-Share:

- What is this? What does it look like?
- Which values does it take?
- How many values can it take?

- Is it an image?

```
>>> import matplotlib.pyplot as plt
>>> I = np.random.rand(256,256);
>>> plt.imshow(I);
>>> plt.show();
```

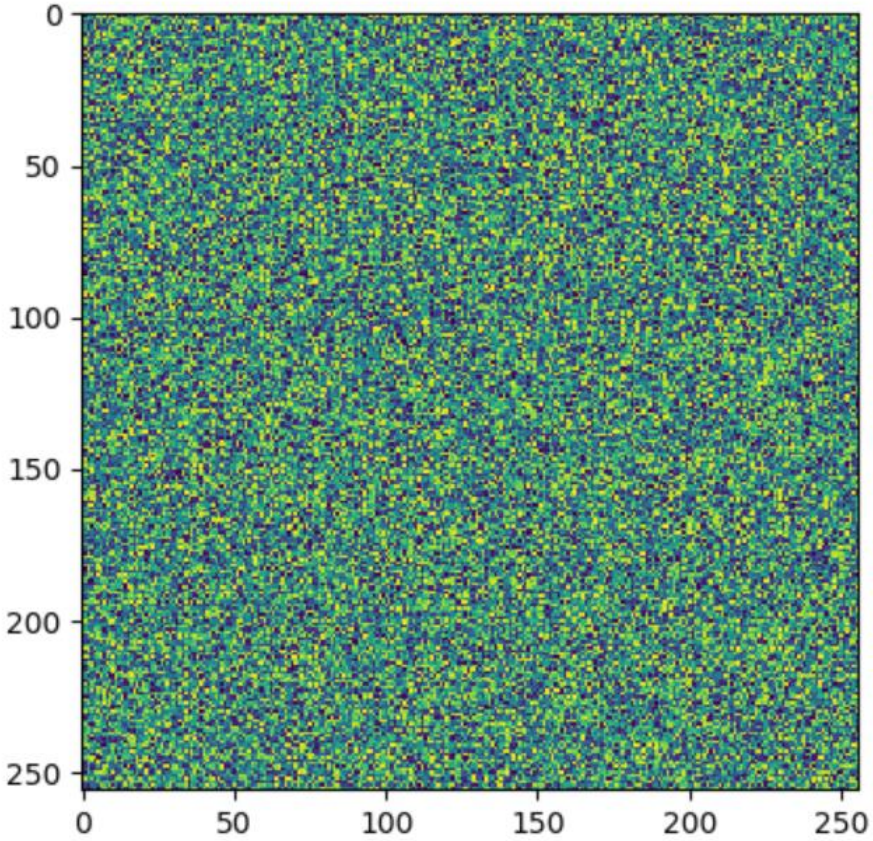


Note: matplotlib and colour maps

```
>>> from matplotlib import pyplot as p
>>> from numpy import random as r
>>> I = r.rand(256,256);
>>> p.imshow(I);
>>> p.show();
```

This code will actually produce an image like this:

matplotlib assumes the data is ‘scientific’ and applies a colourmap.

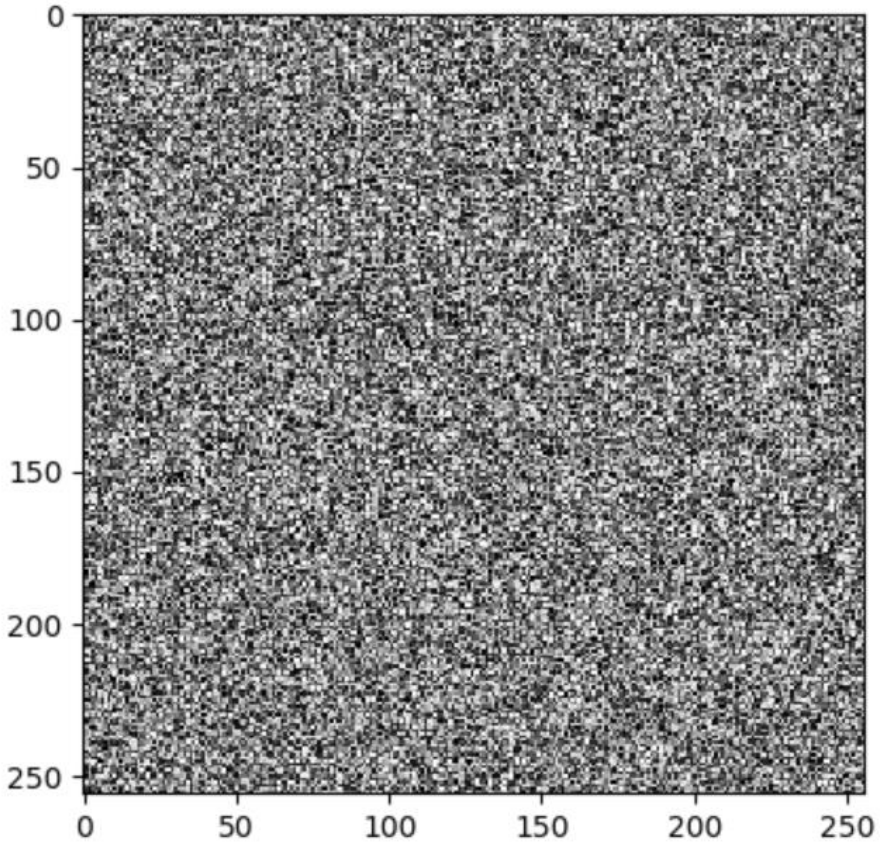


[Additional info]

Note: matplotlib and colour maps

```
>>> from matplotlib import pyplot as p
>>> from numpy import random as r
>>> I = r.rand(256,256);
>>> p.imshow(I,cmap='gray');
>>> p.show();
```

Better.



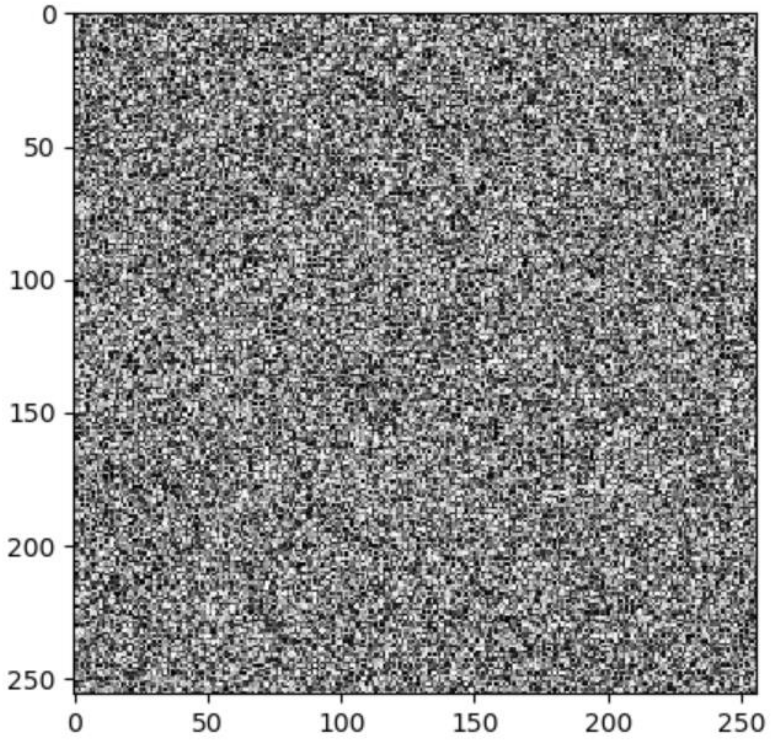
[Additional info]

Note: matplotlib and colour maps

```
>>> from matplotlib import pyplot as p
>>> from numpy import random as r
>>> I = r.rand(256,256) * 0.5;
>>> p.imshow(I,cmap='gray');
>>> p.show();
```

Next issue:

matplotlib will normalize input
e.g., map the lowest value to 0,
and the highest to 1.
Even though we multiplied by 0.5,
the overall brightness is the same.

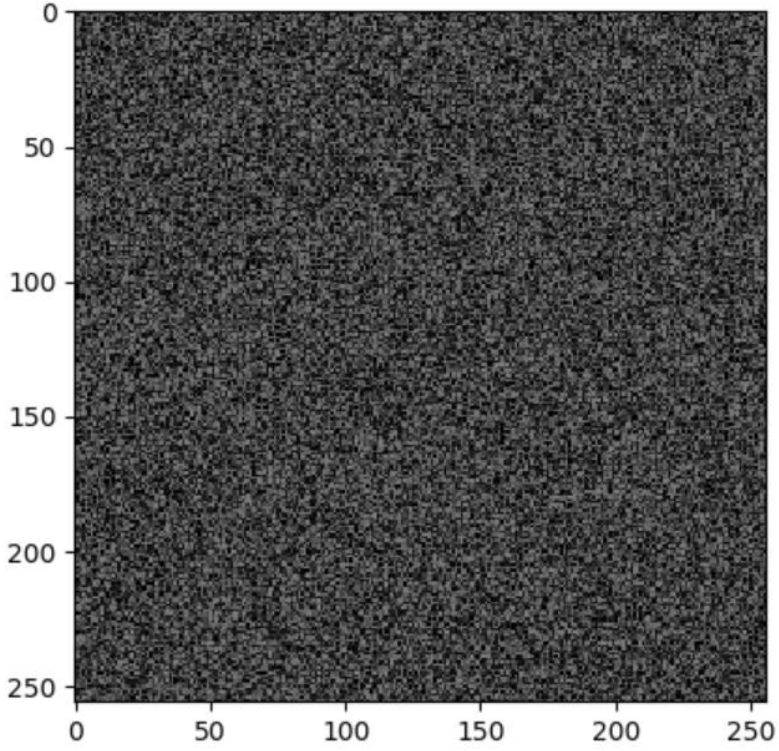


[Additional info]

Note: matplotlib and colour maps

```
>>> from matplotlib import pyplot as p
>>> from numpy import random as r
>>> I = r.rand(256,256) * 0.5;
>>> p.imshow(I,cmap='gray',vmin=0.0,vmax=1.0);
>>> p.show();
```

Better!



[Additional info]

Dimensionality of an Image

- @ 8bit = 256 values ^ 65,536
  - Computer says ‘Inf’ combinations.

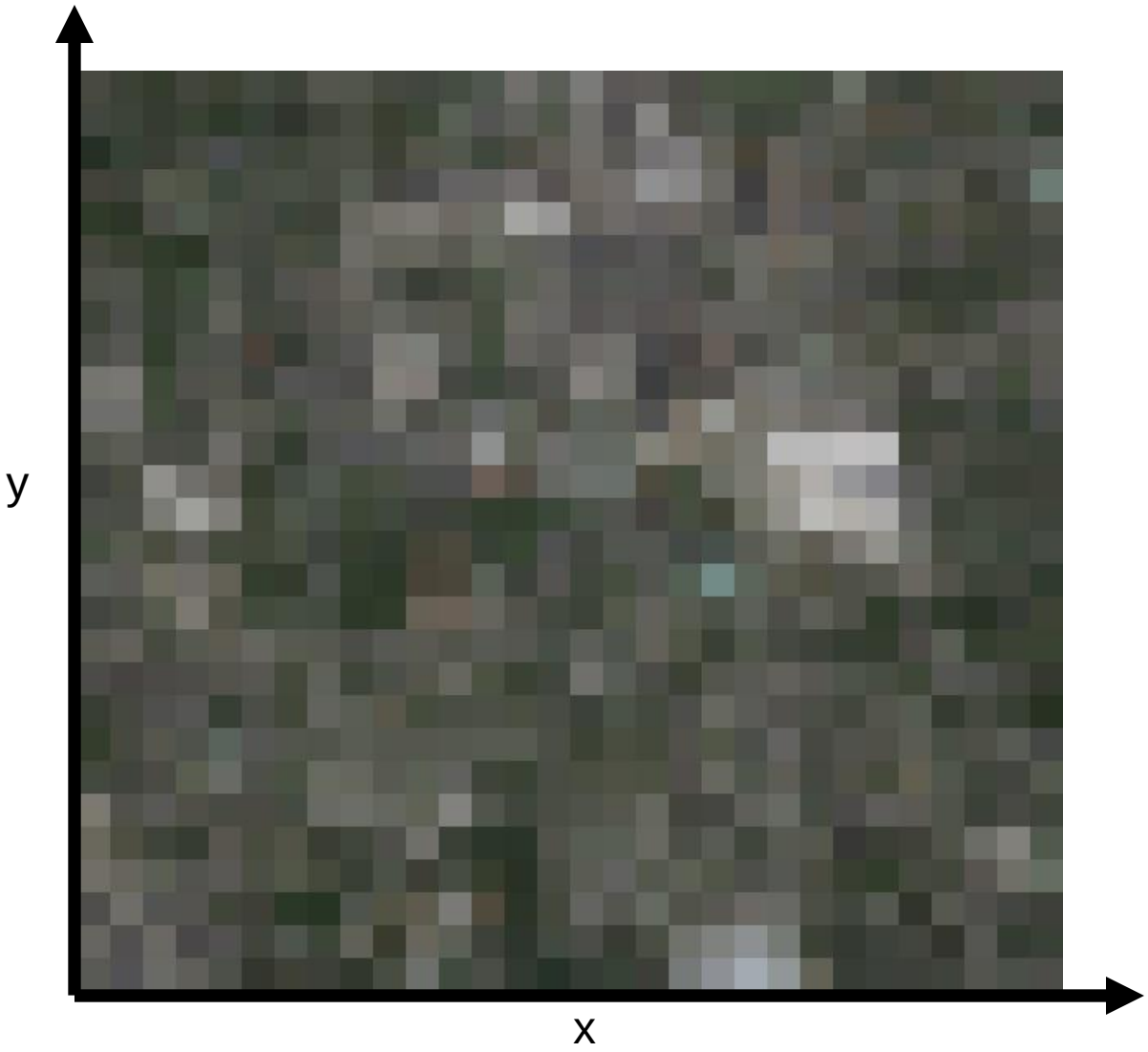
- Some depiction of all possible scenes would fit into this memory.

Dimensionality of an Image

- @ 8bit = 256 values ^ 65,536
  - Computer says ‘Inf’ combinations.

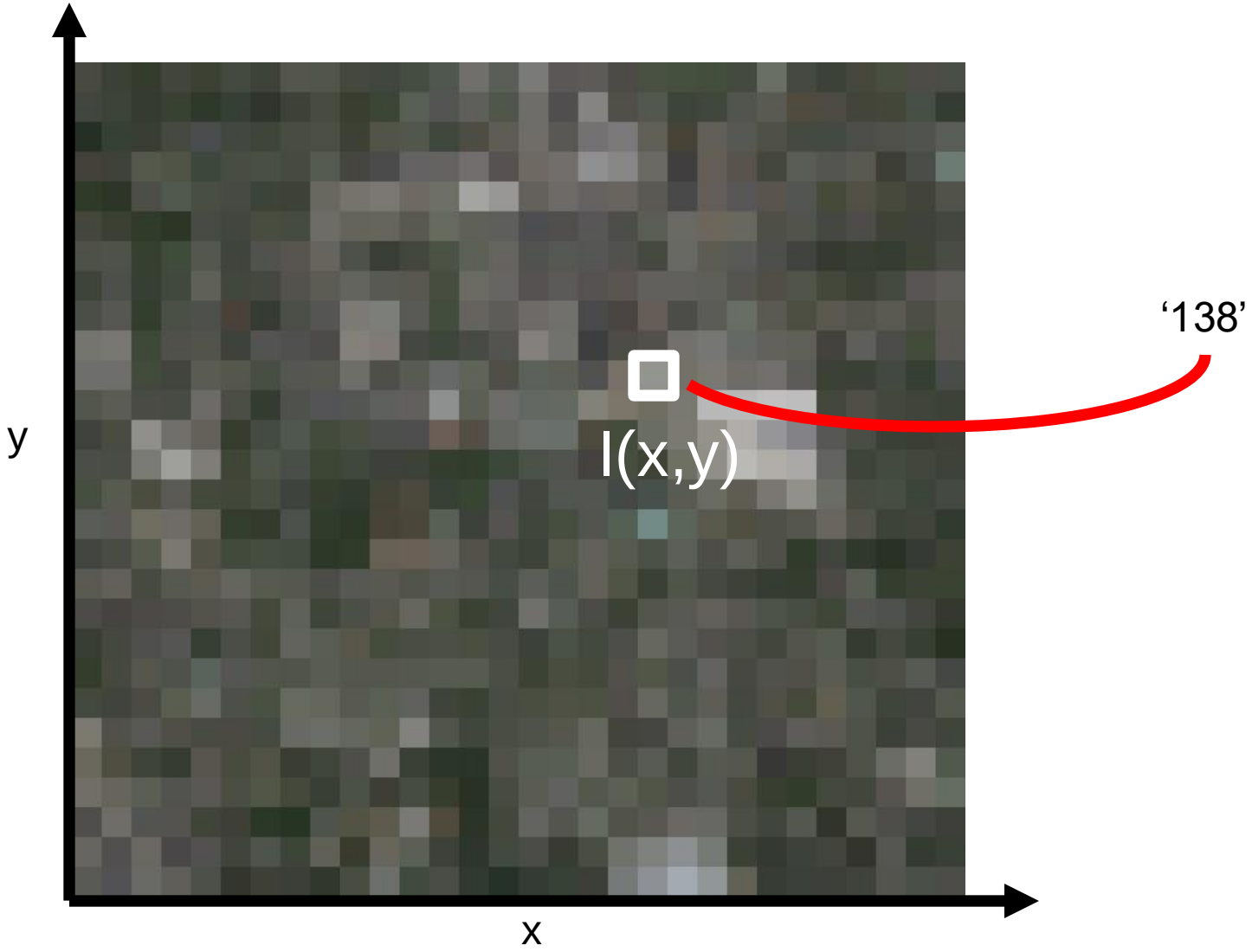
- Some depiction of all possible scenes would fit into this memory.
- Computer vision as making sense of an extremely high-dimensional space.
  - Subspace of ‘natural’ images.
  - Deriving low-dimensional, explainable models.

What is each part of an image?

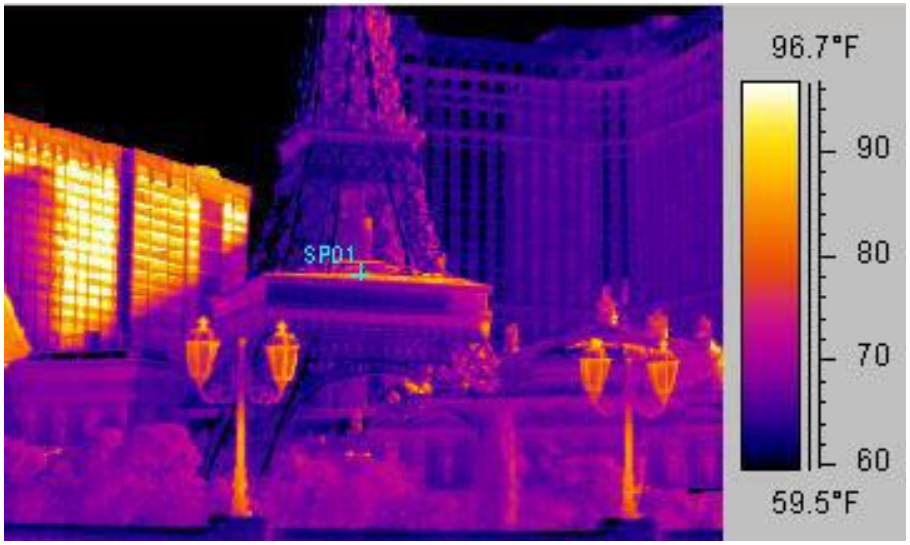


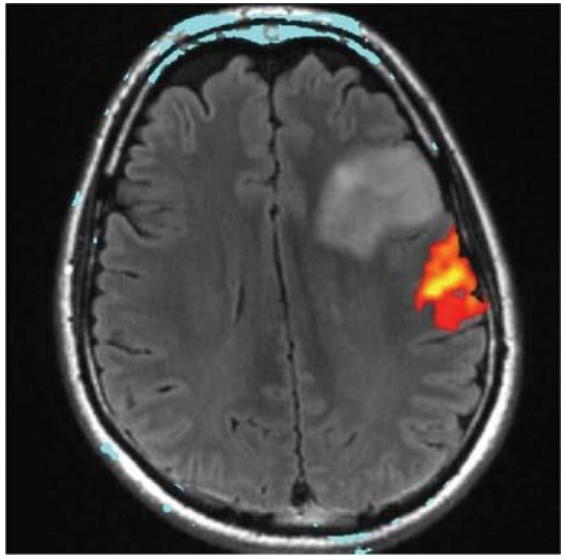
What is each part of an image?

- Pixel -> picture element

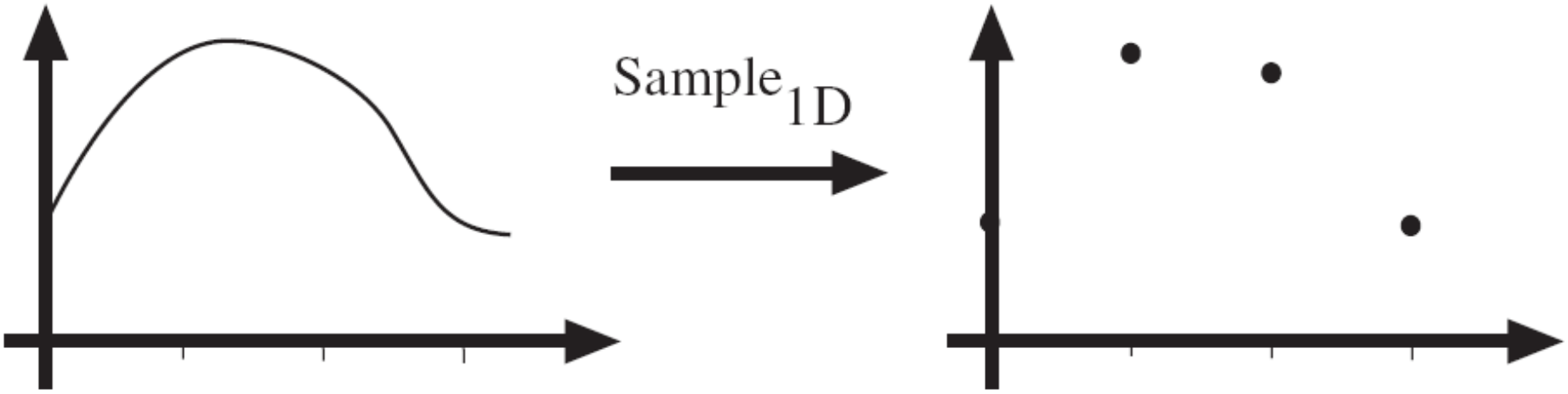


Example 2D Images



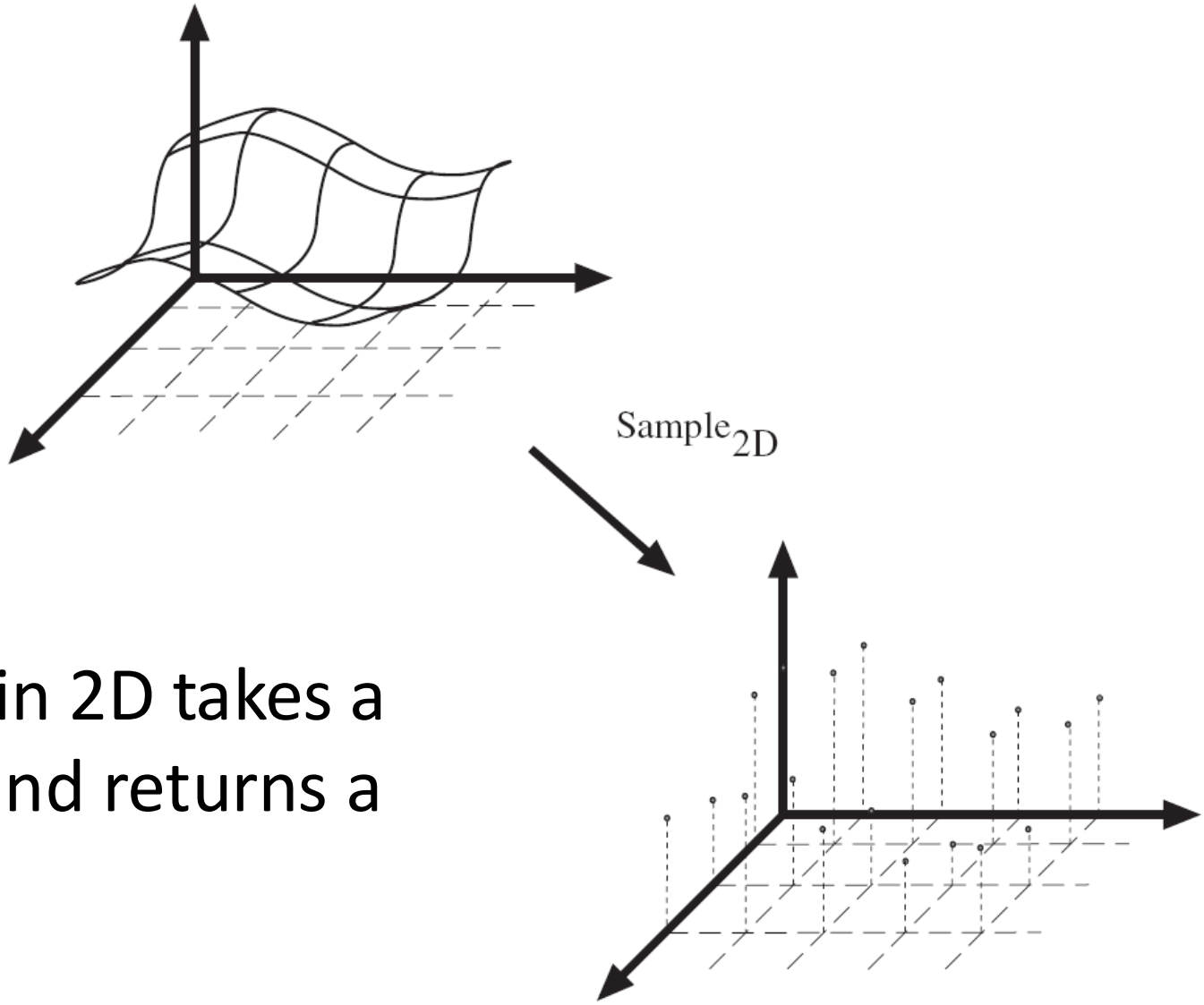


Sampling in 1D



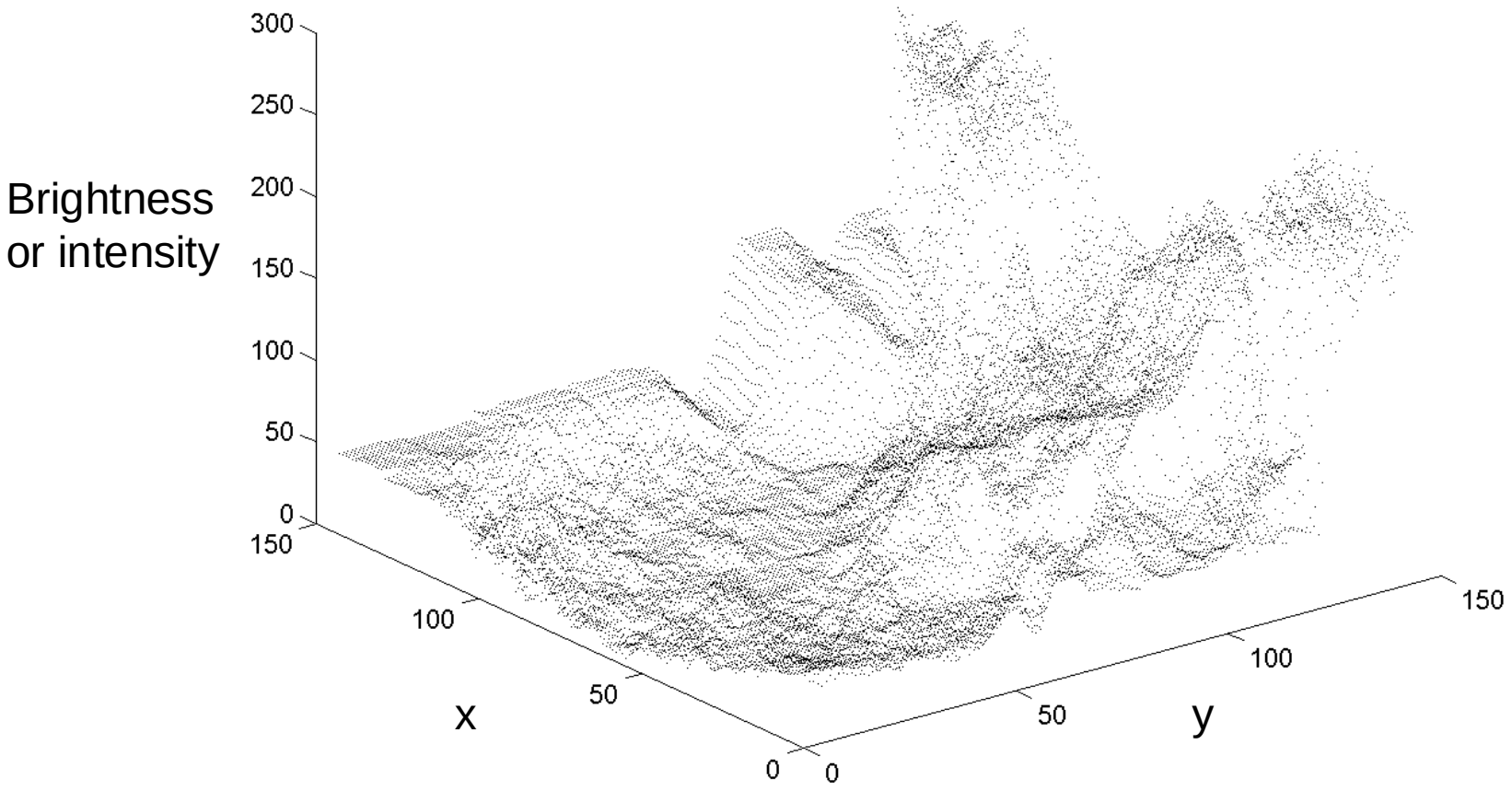
- Sampling in 1D takes a function, and returns a vector whose elements are values of that function at the sample points.

Sampling in 2D



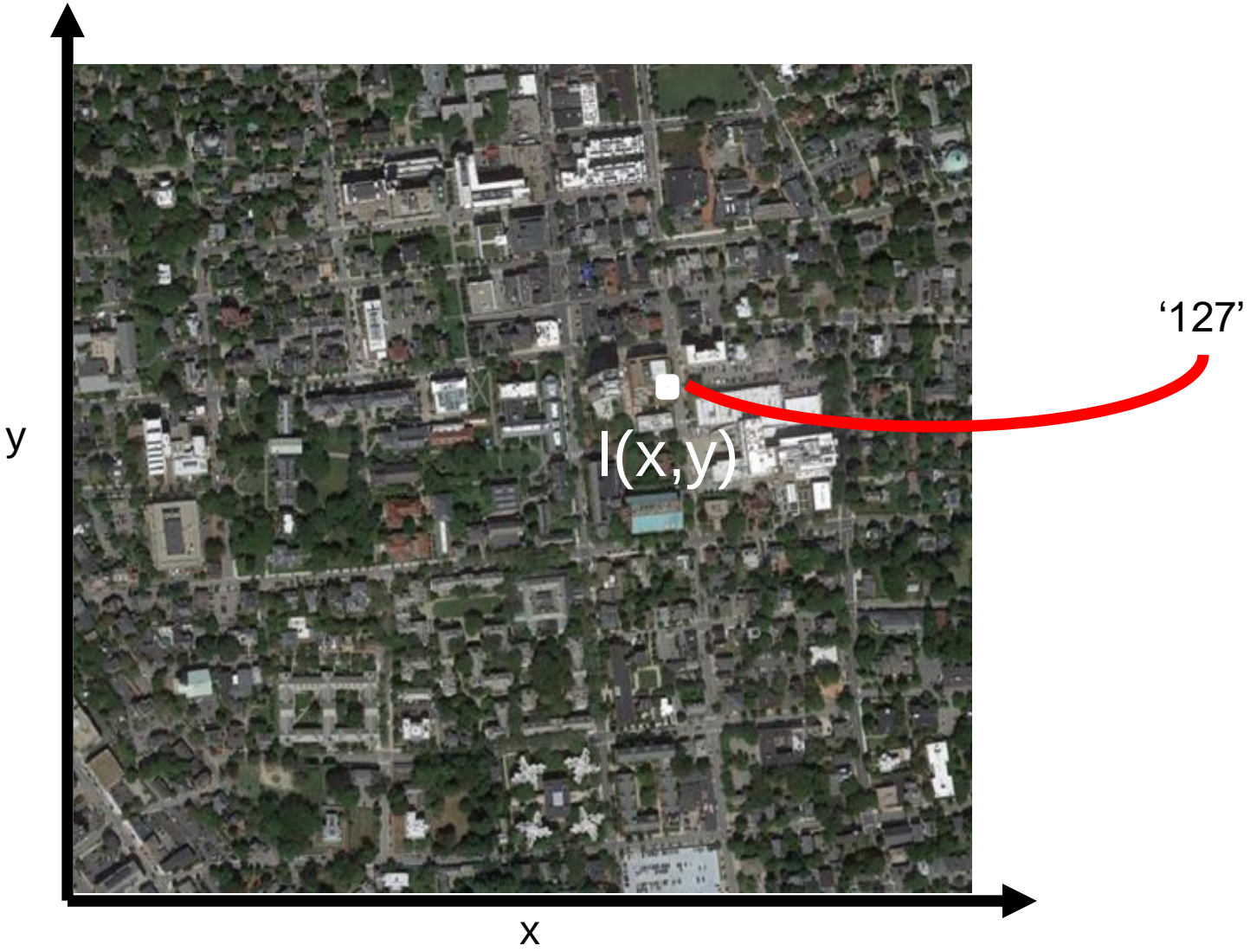
- Sampling in 2D takes a function and returns a matrix.

Grayscale Digital Image



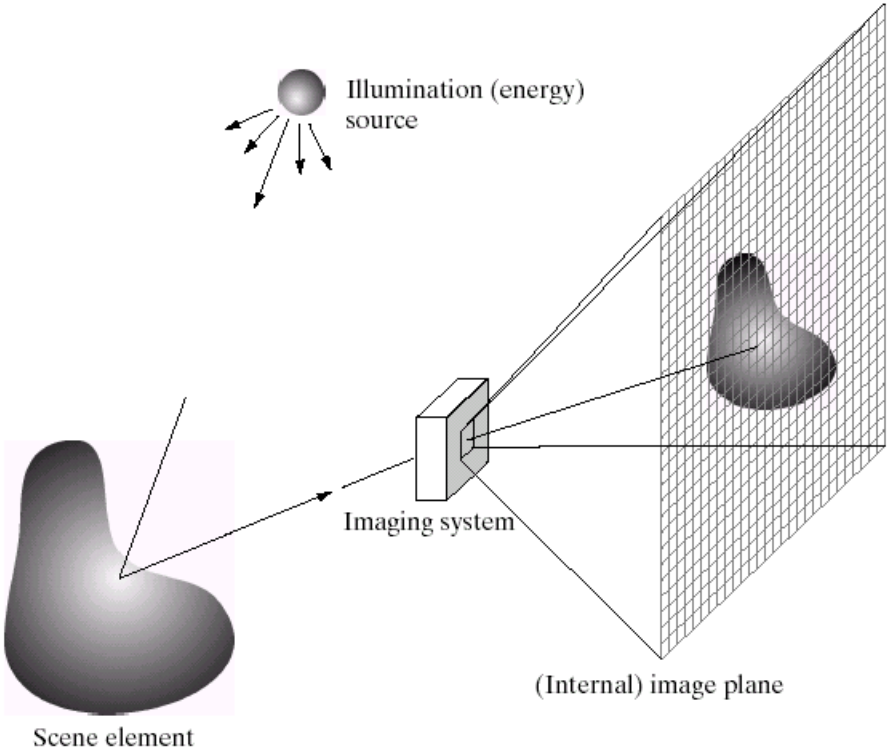
What is each part of a photograph?

- Pixel -> picture element

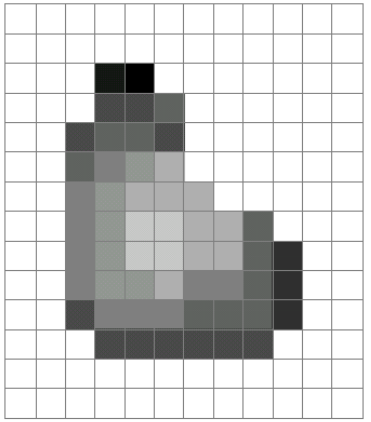


What is an image?

Integrating light over a range of angles.



Camera Sensor

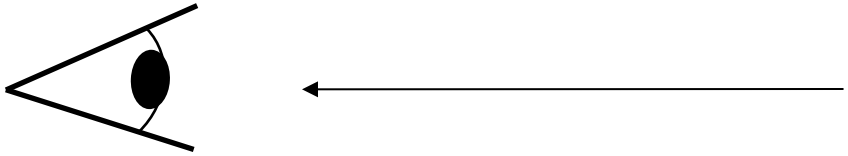


Output Image

What is light?

Electromagnetic radiation (EMR) moving along rays in space

- R(λ) is EMR, measured in units of power (watts)
  - λ is wavelength

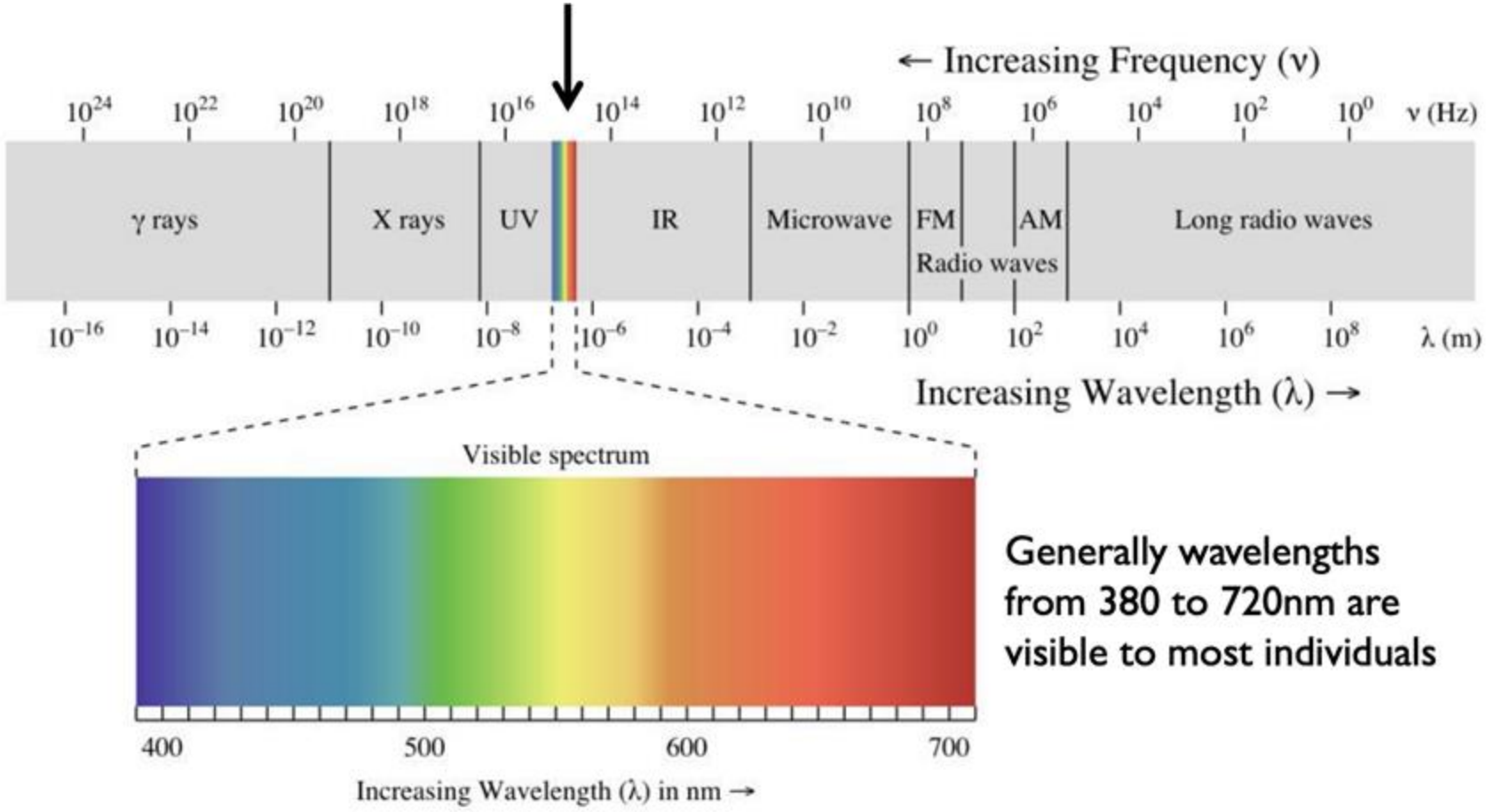


Perceiving light

- How do we convert radiation into “color”?
- What part of the spectrum do we see?

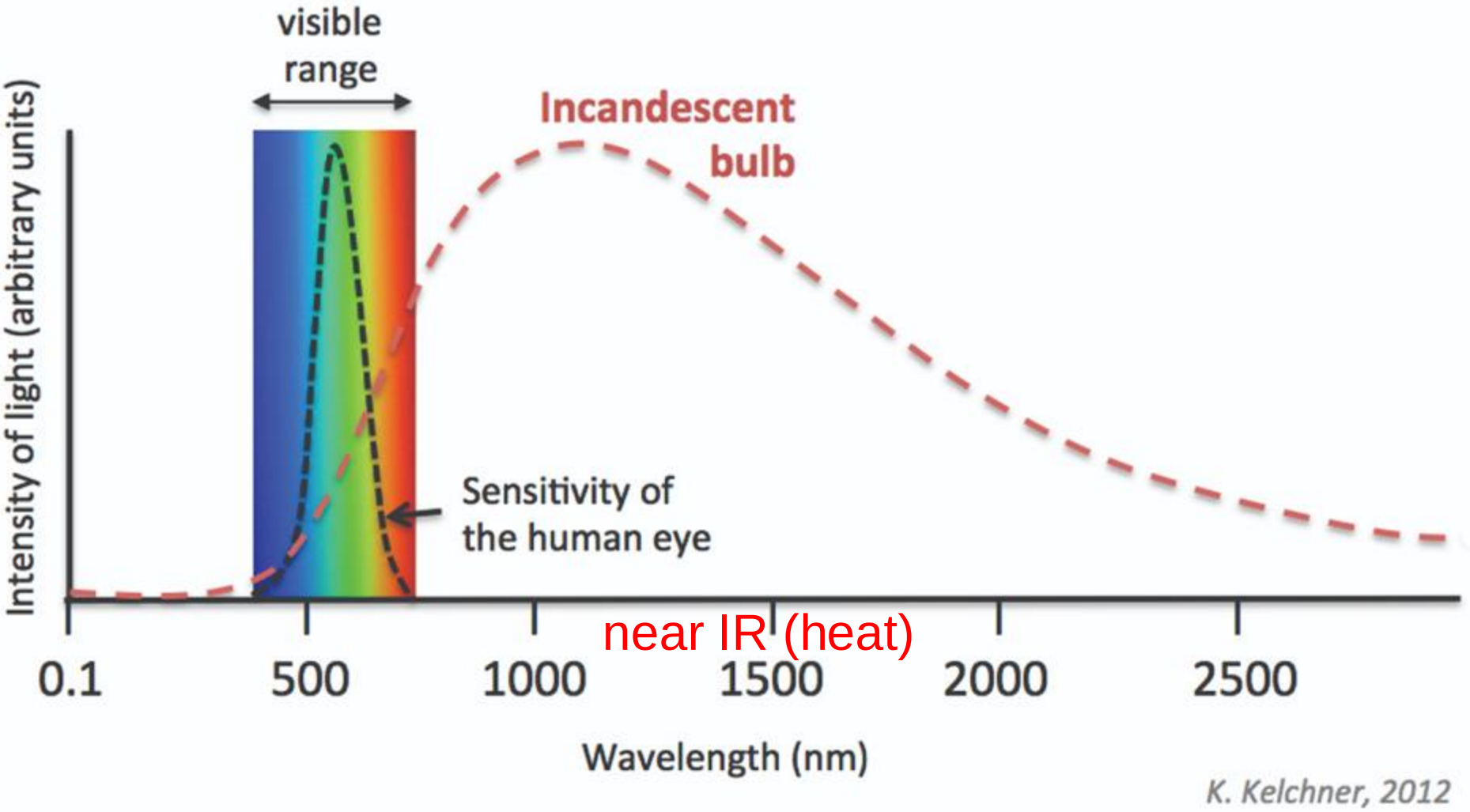
The visible light spectrum

We “see” electromagnetic radiation in a small range of wavelengths



Generally wavelengths from 380 to 720nm are visible to most individuals

from Michael Brown, ICCV19 color tutorial


visible
range
Incandescent
bulb
Intensity of light (arbitrary units)
Sensitivity of
the human eye
near IR (heat)
0.1
500
1000
1500
2000
2500
Wavelength (nm)
K. Kelchner, 2012

SPD of different light sources

The appearance of light depends on its SPD

- How much power (or energy) at each wavelength

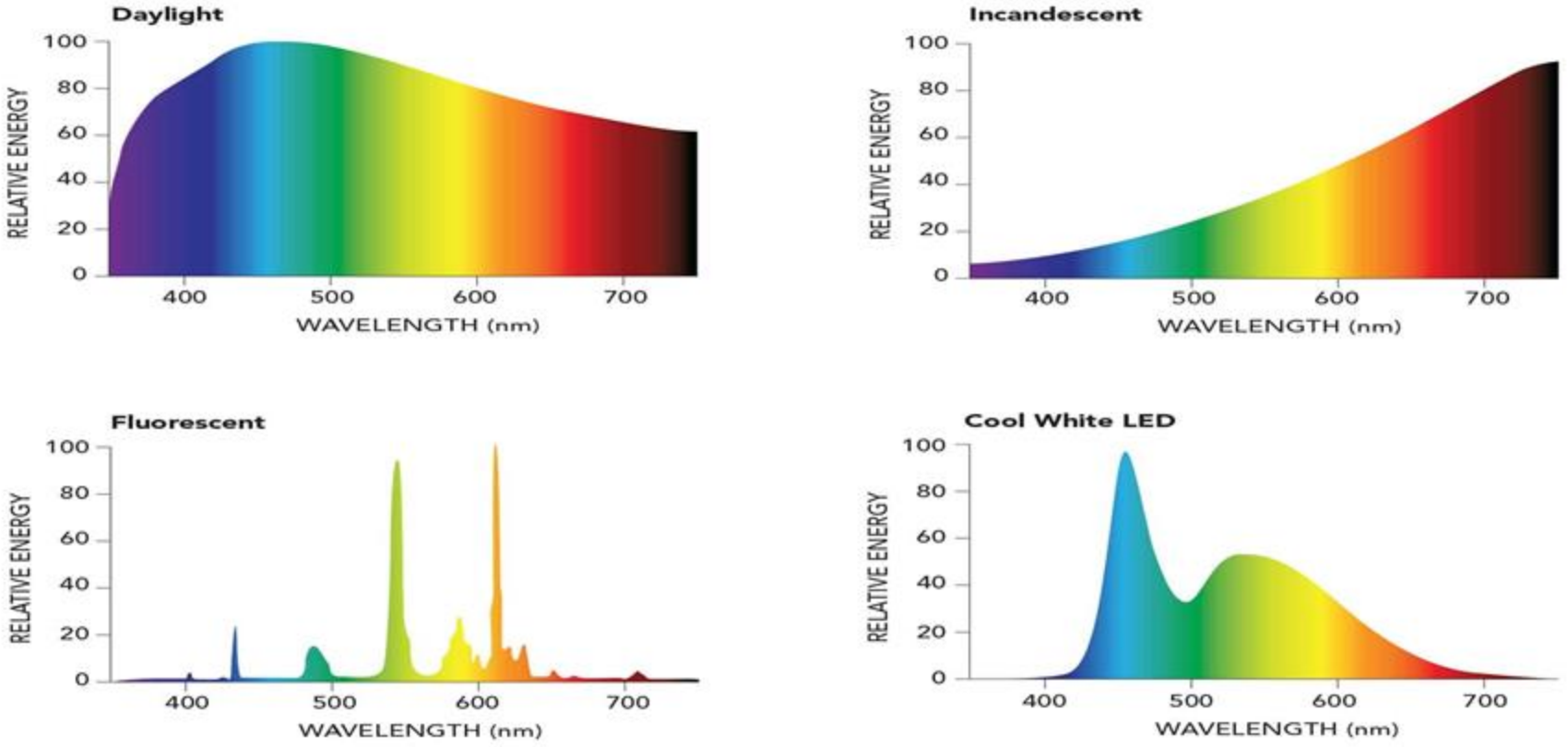


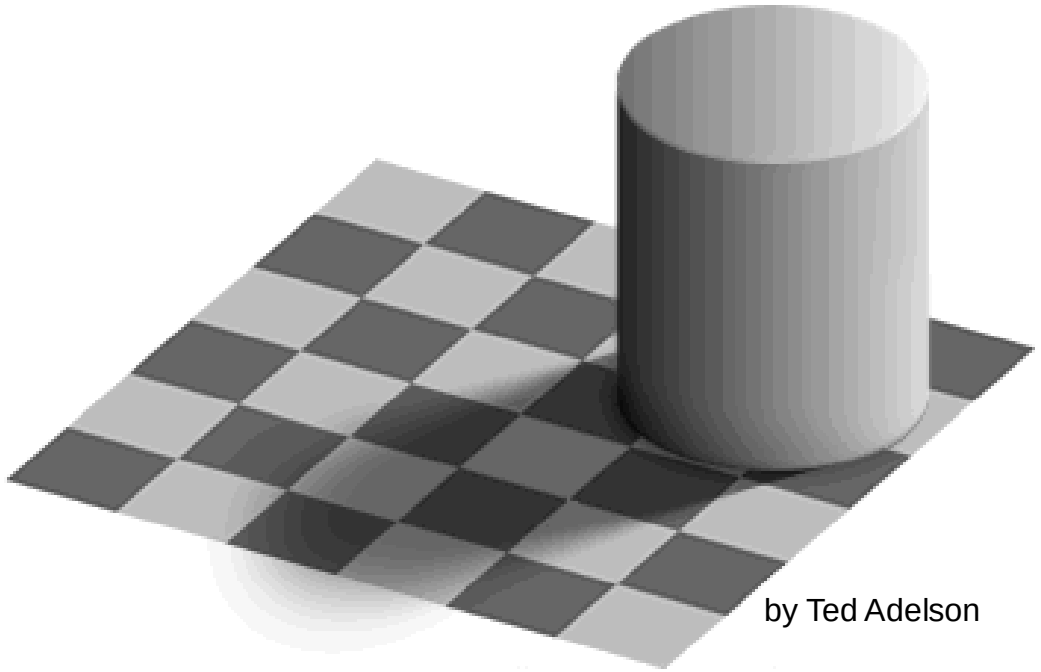
Figure 2-3. Spectral distribution of typical daylight, incandescent, fluorescent and LED. Note the distinct blue spike characteristic of LED.

Demonstrations of visual acuity

With left eye shut, look at the cross on the left. At the right distance, the circle on the right should disappear (Glassner, 1.8).

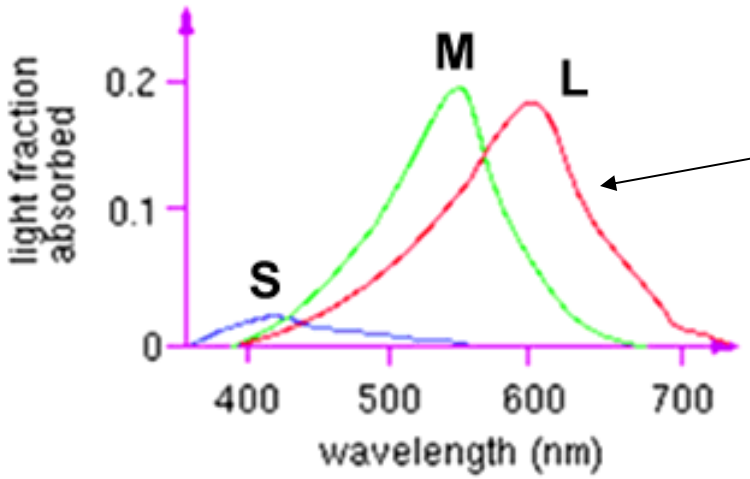
Brightness contrast

The apparent brightness depends on the surrounding region



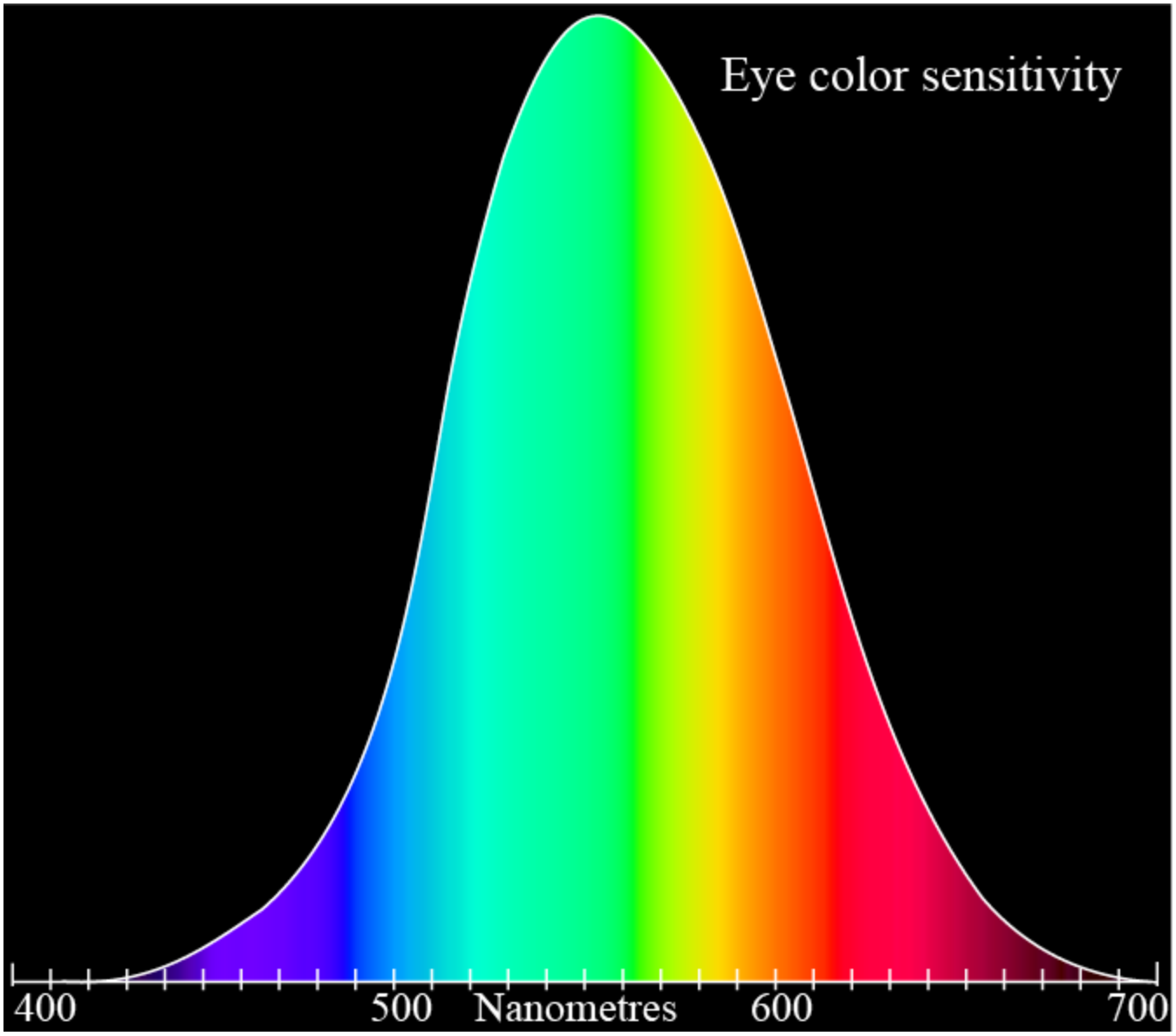
by Ted Adelson

Color perception

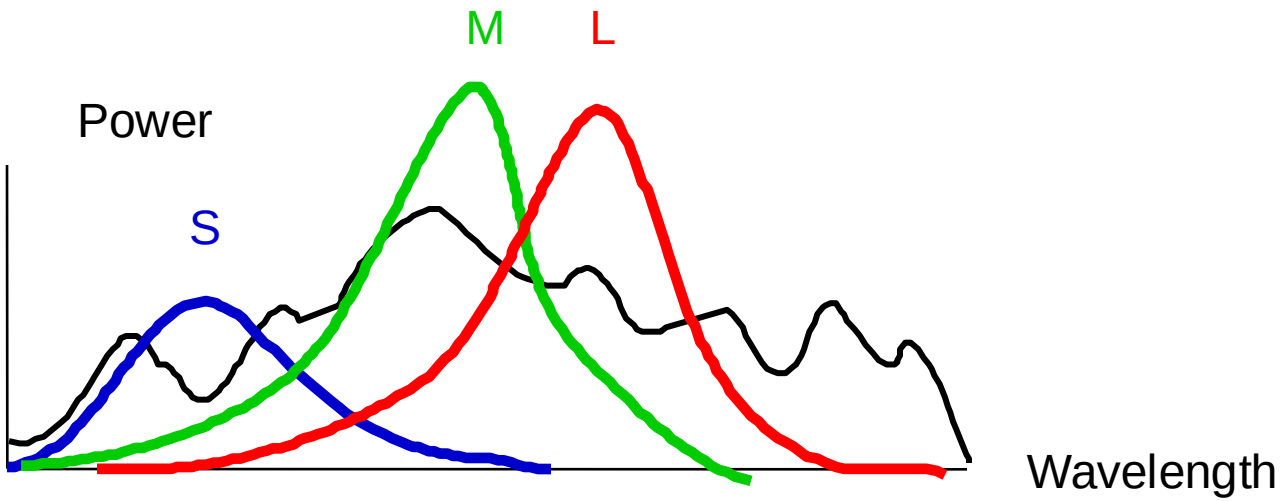


Three types of cones

- Each is sensitive in a different region of the spectrum
  - but regions overlap
  - Short (S) corresponds to blue
  - Medium (M) corresponds to green
  - Long (L) corresponds to red
- Different sensitivities: we are more sensitive to green than red
  - varies from person to person (and with age)
- Colorblindness—deficiency in at least one type of cone


Eye color sensitivity
400
500
Nanometres
600
700

Color perception



Rods and cones act as filters on the spectrum (SPD)

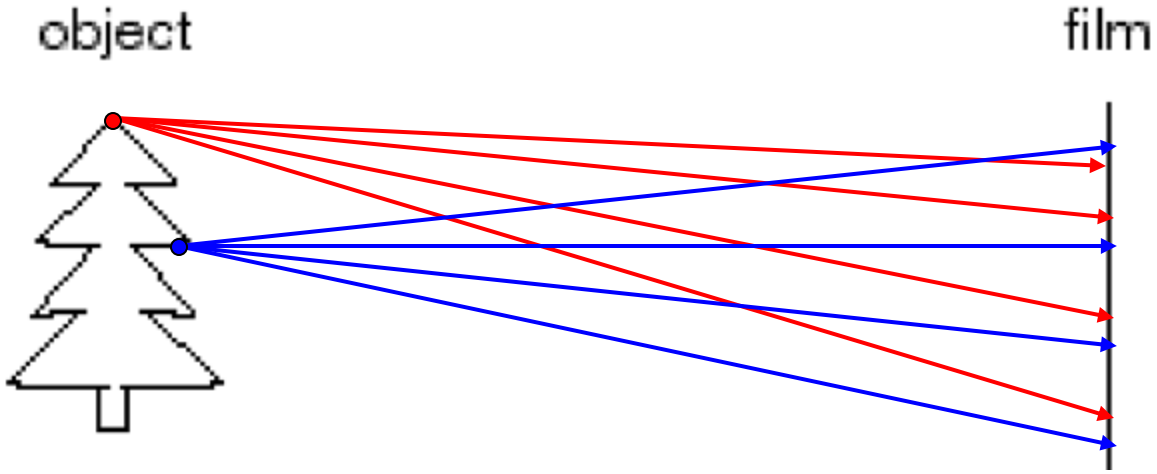
- To get the output of a filter, multiply its response curve by the spectrum, integrate over all wavelengths
  - Each cone yields one number
- Q: How can we represent an entire spectrum with 3 numbers?
- A: We can't! Most of the information is lost.

– As a result, two different spectra may appear indistinguishable

» such spectra are known as metamers

» https://cs.brown.edu/courses/cs123/demos/metamers/index.html

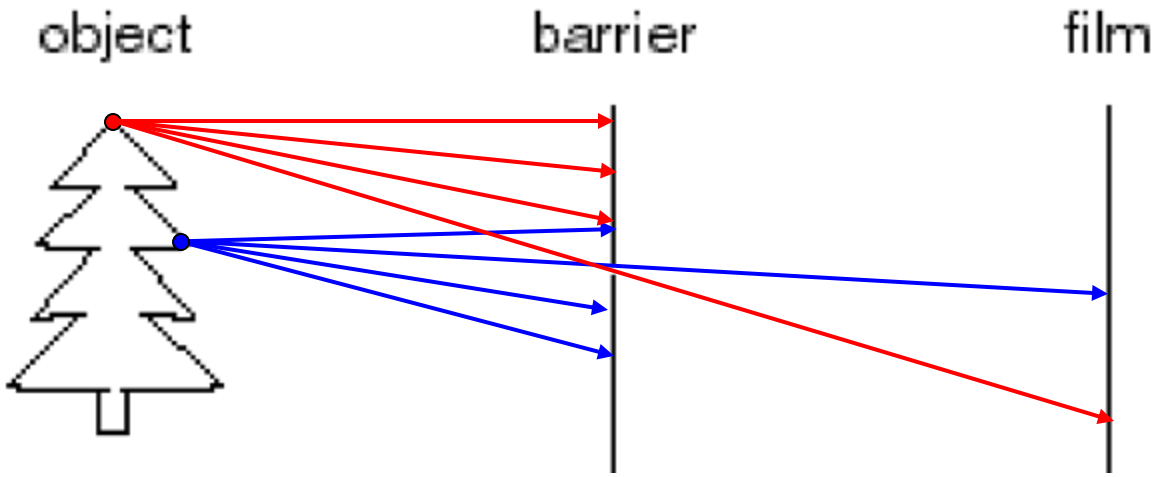
Image formation



Let’s design a camera

- Idea 1: put a piece of film in front of an object
- Do we get a reasonable image?

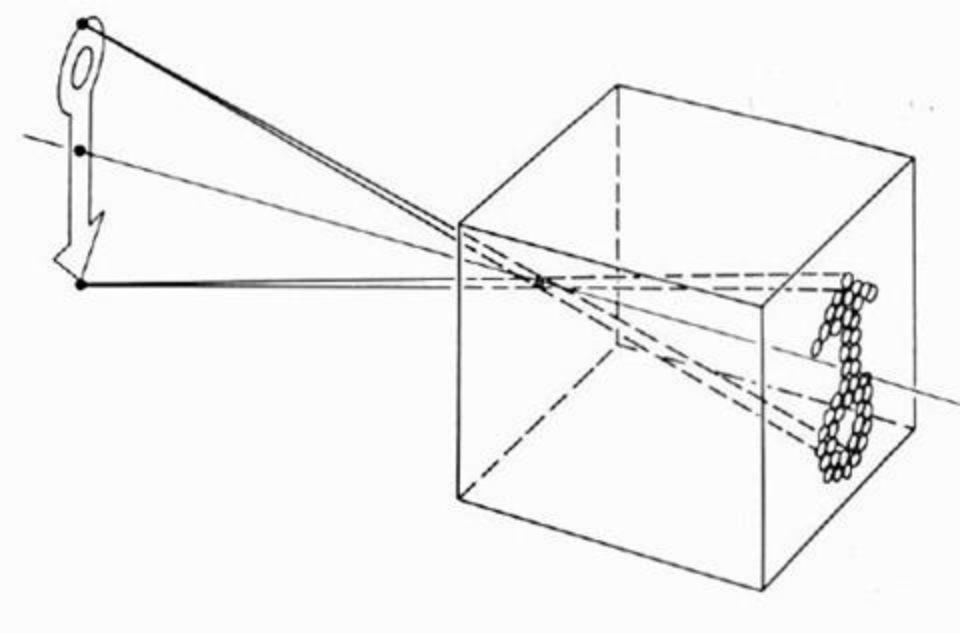
Pinhole camera



Add a barrier to block off most of the rays

- This reduces blurring
- The opening known as the aperture
- How does this transform the image?

Pinhole Camera (*Camera Obscura*)



The first camera

- Known to Aristotle

Pinhole cameras everywhere

Sun “shadows” during a solar eclipse

by Henrik von Wendt http://www.flickr.com/photos/hvw/2724969199/

Pinhole cameras everywhere

Sun “shadows” during a solar eclipse

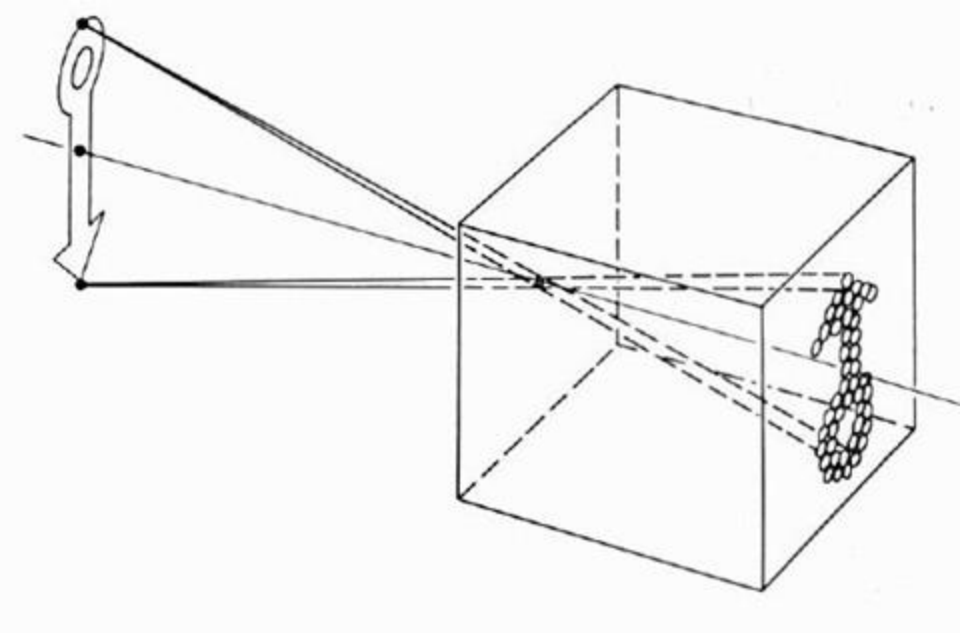
http://www.flickr.com/photos/73860948@N08/6678331997/

Pinhole cameras everywhere

Tree shadow during a solar eclipse

photo credit: Nils van der Burg
http://www.physicstogo.org/index.cfm

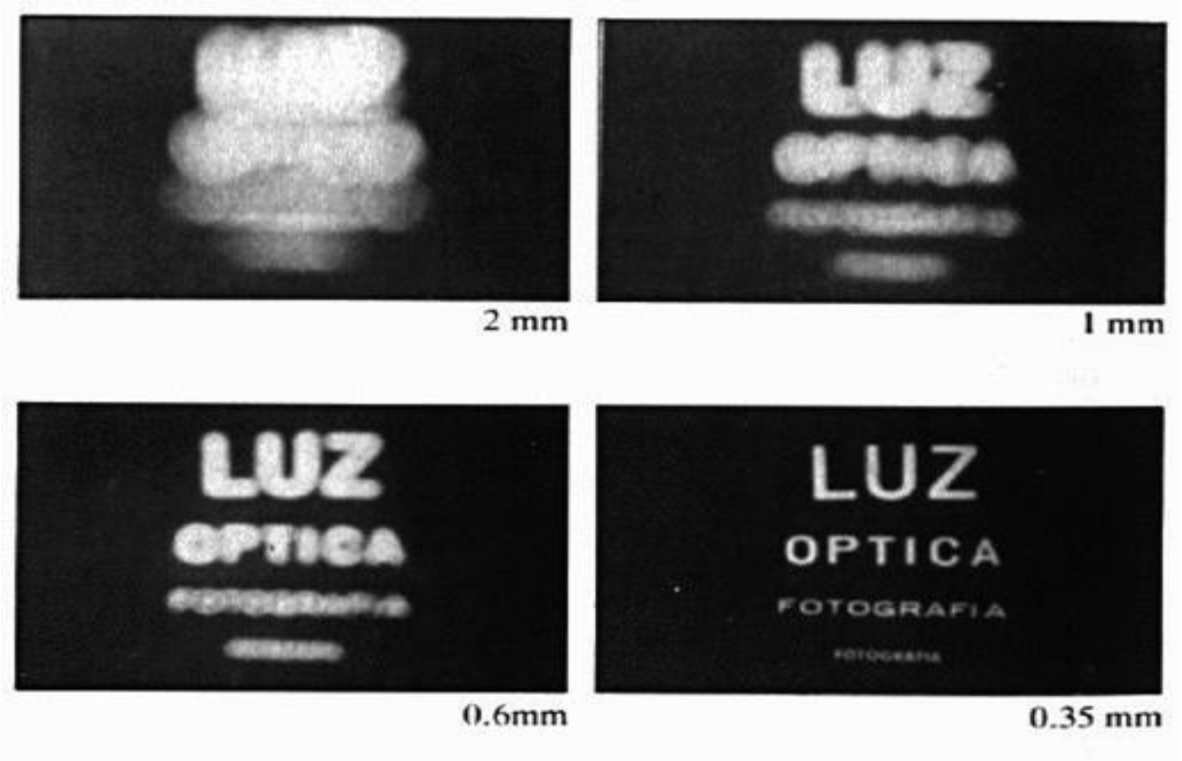
Camera Obscura



The first camera

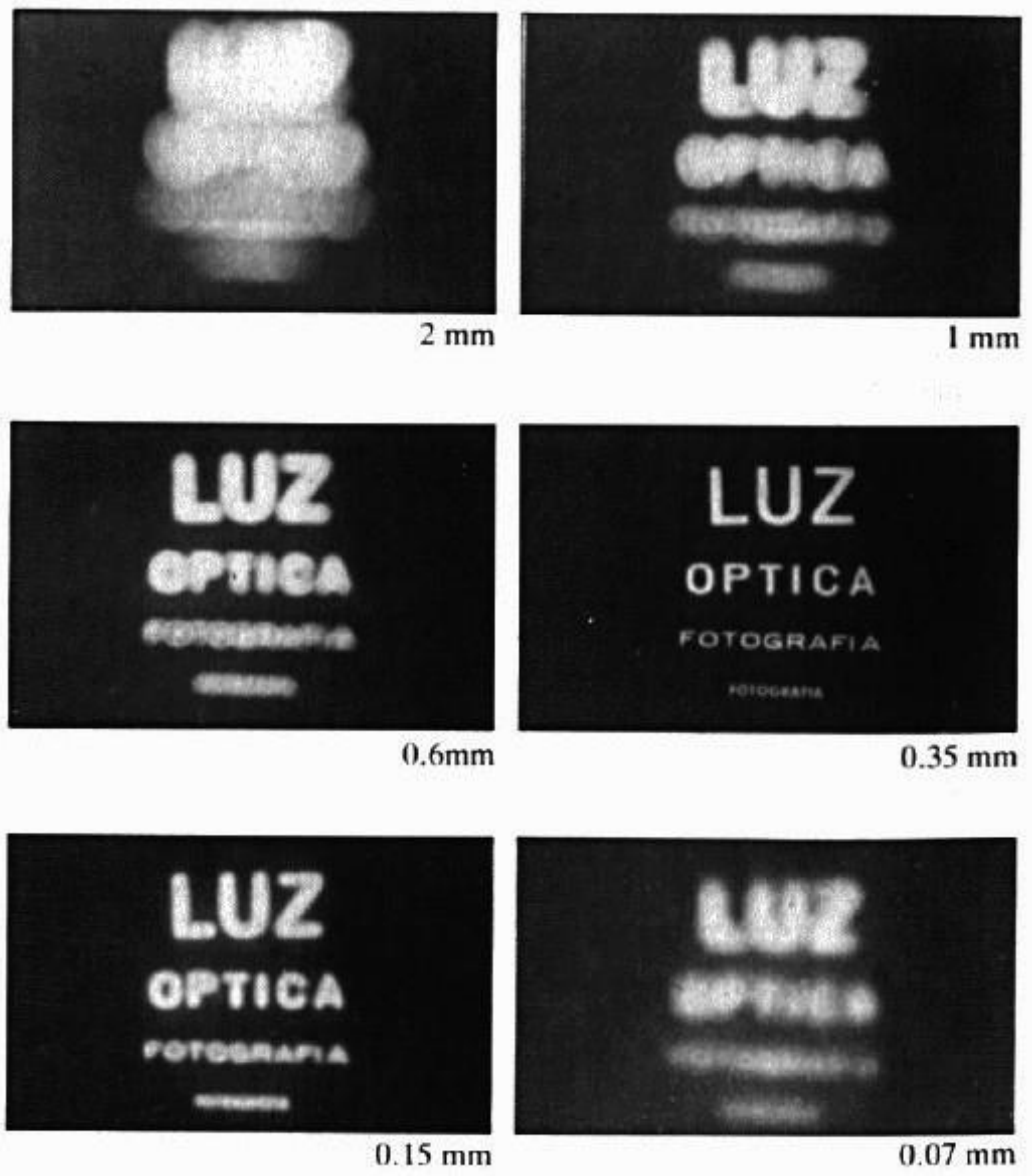
- Known to Aristotle
- How does the aperture size affect the image?

Shrinking the aperture

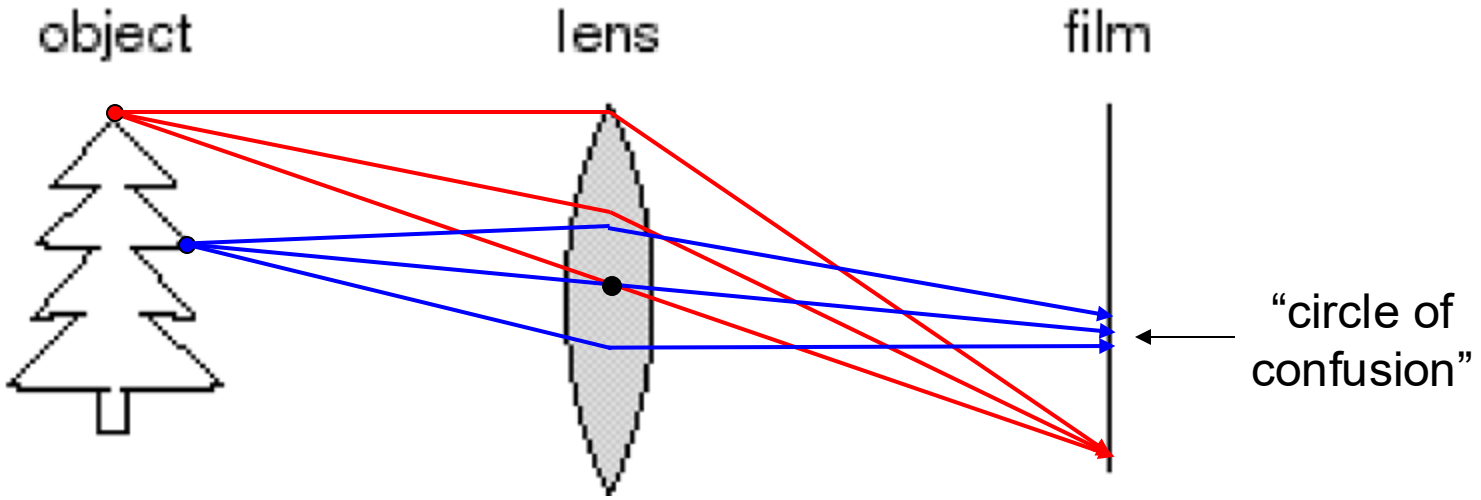


Why not make the aperture as small as possible?

Shrinking the aperture



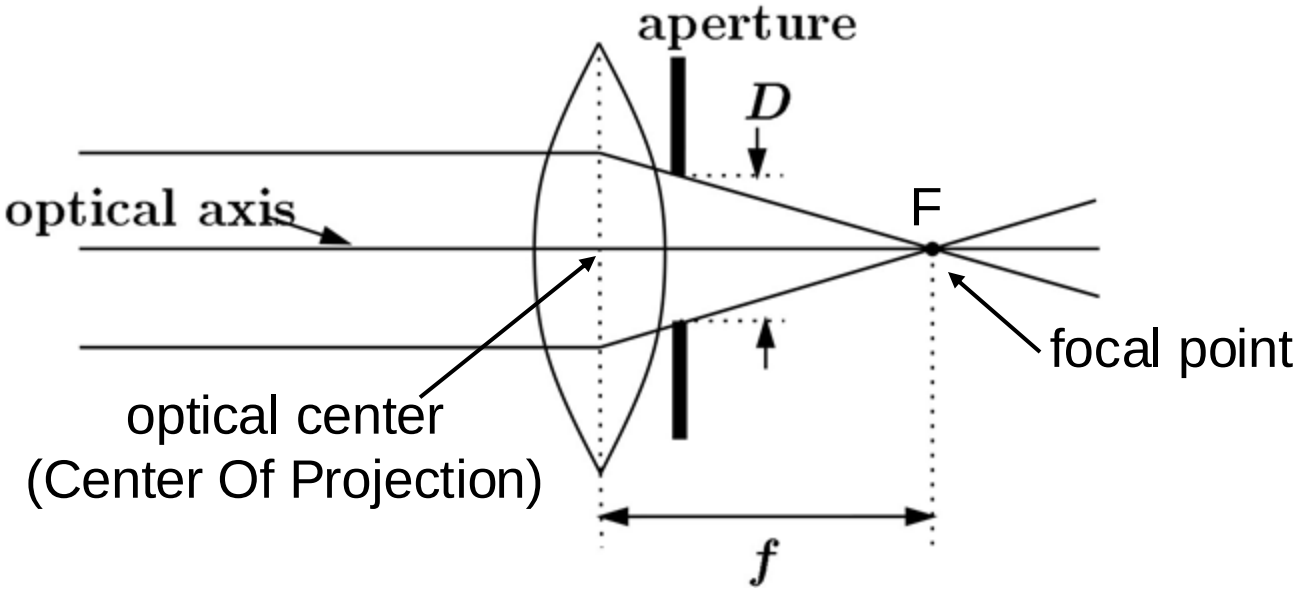
Adding a lens



A lens focuses light onto the film

- There is a specific distance at which objects are “in focus”
  - other points project to a “circle of confusion” in the image
- Changing the shape of the lens changes this distance

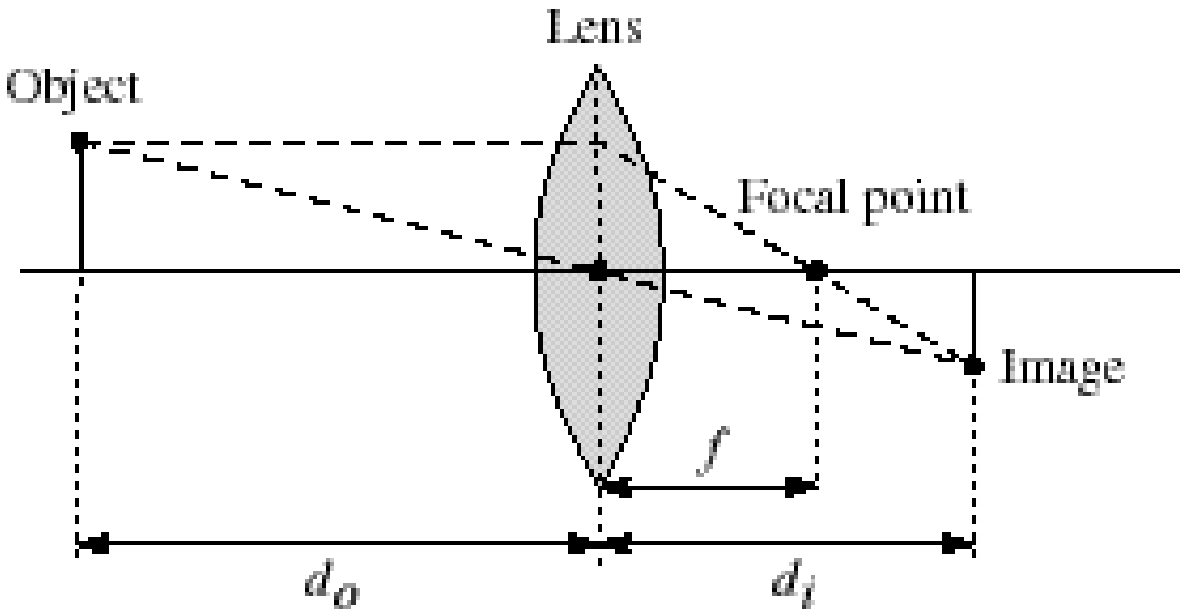
Lenses



A lens focuses parallel rays onto a single focal point

- focal point at a distance f beyond the plane of the lens
  - f is a function of the shape and index of refraction of the lens
- Aperture of diameter D restricts the range of rays
  - aperture may be on either side of the lens
- Lenses are typically spherical (easier to produce)

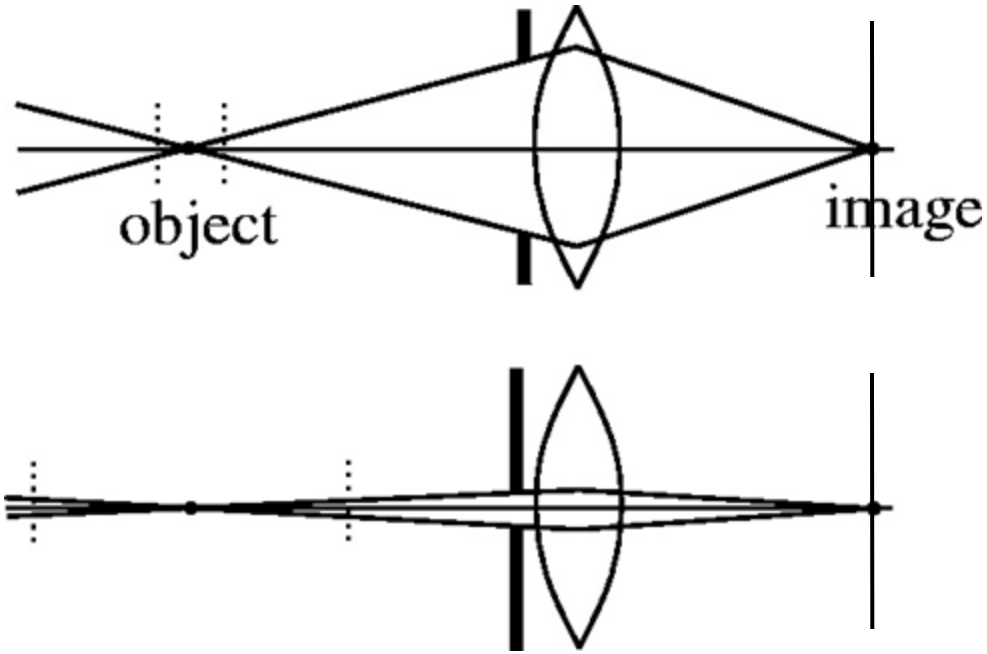
Thin lenses



Thin lens equation: $$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

- Any object point satisfying this equation is in focus
- What is the shape of the focus region?
- How can we change the focus region?
- Thin lens applet: http://www.phy.ntnu.edu.tw/java/Lens/lens_e.html (by Fu-Kwun Hwang)

Depth of field



f / 5.6

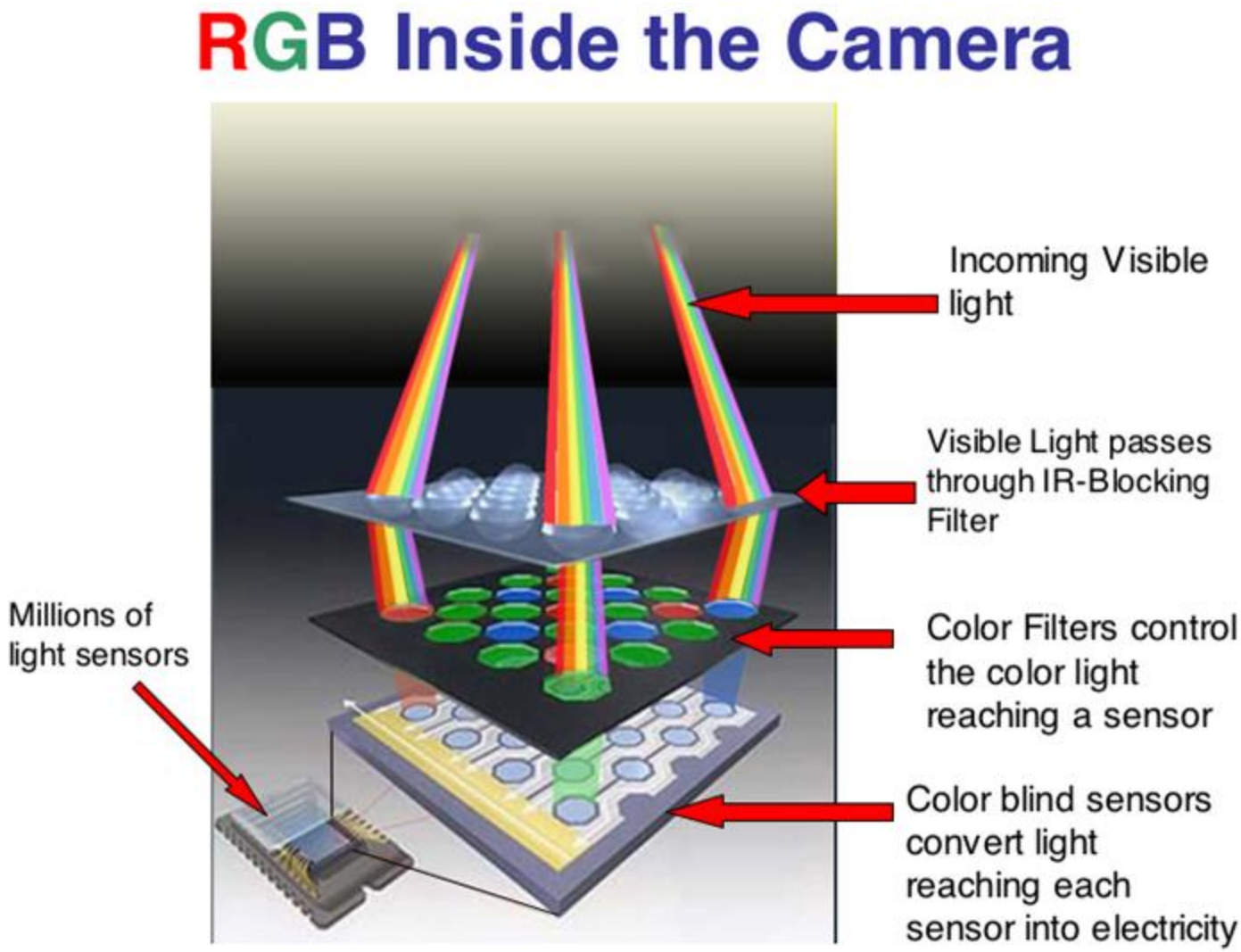
f / 32

Changing the aperture size affects depth of field

- A smaller aperture increases the range in which the object is approximately in focus

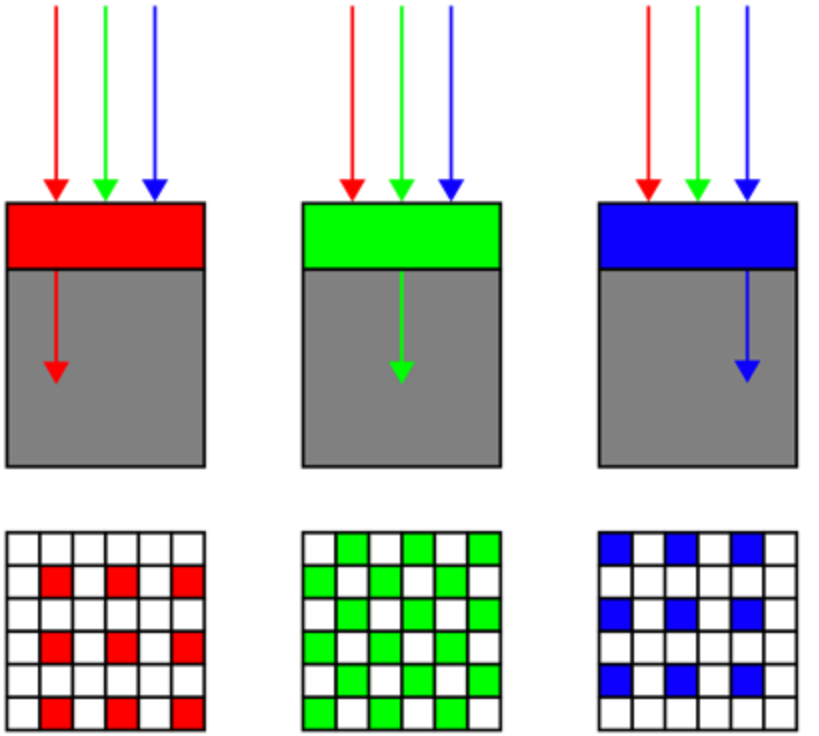
Flower images from Wikipedia http://en.wikipedia.org/wiki/Depth_of_field

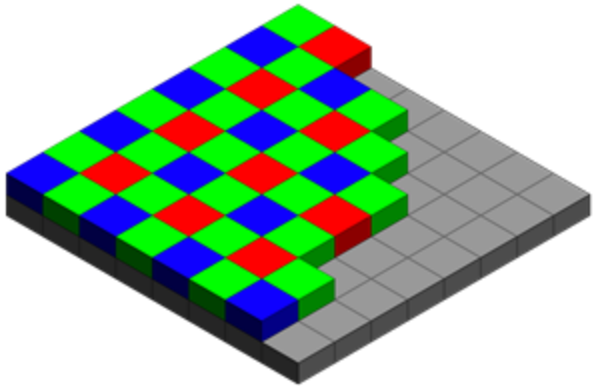
From light to pixels



https://sites.google.com/a/globalsystemsscience.org/digital-earth-watch/tools/digital-cameras-overview/what-happens-to-

Bayer filters



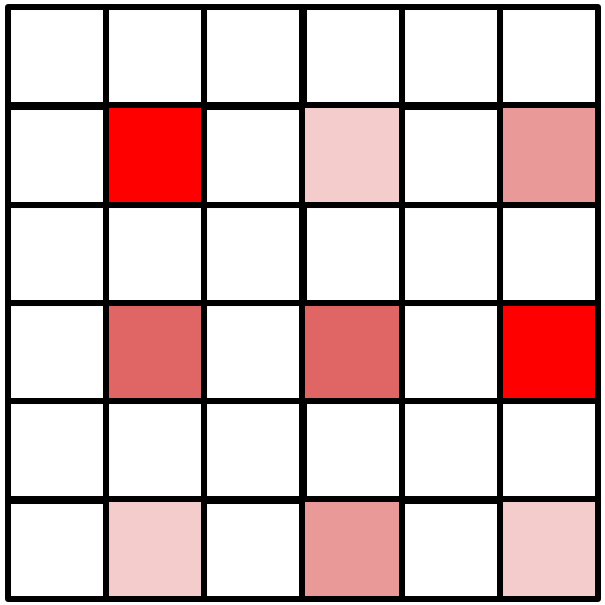


https://en.wikipedia.org/wiki/Bayer_filter

¼ of pixels see red light (e.g.)

- Q: how do you get red at every pixel?
- A: Need to interpolate -- called *debayering*

Debayering



	100		10		30
	50		50		100
	10		30		10

¼ of pixels see red light (e.g.)

- Q: how do you get red at every pixel?
- A: Need to interpolate -- called *debayering*

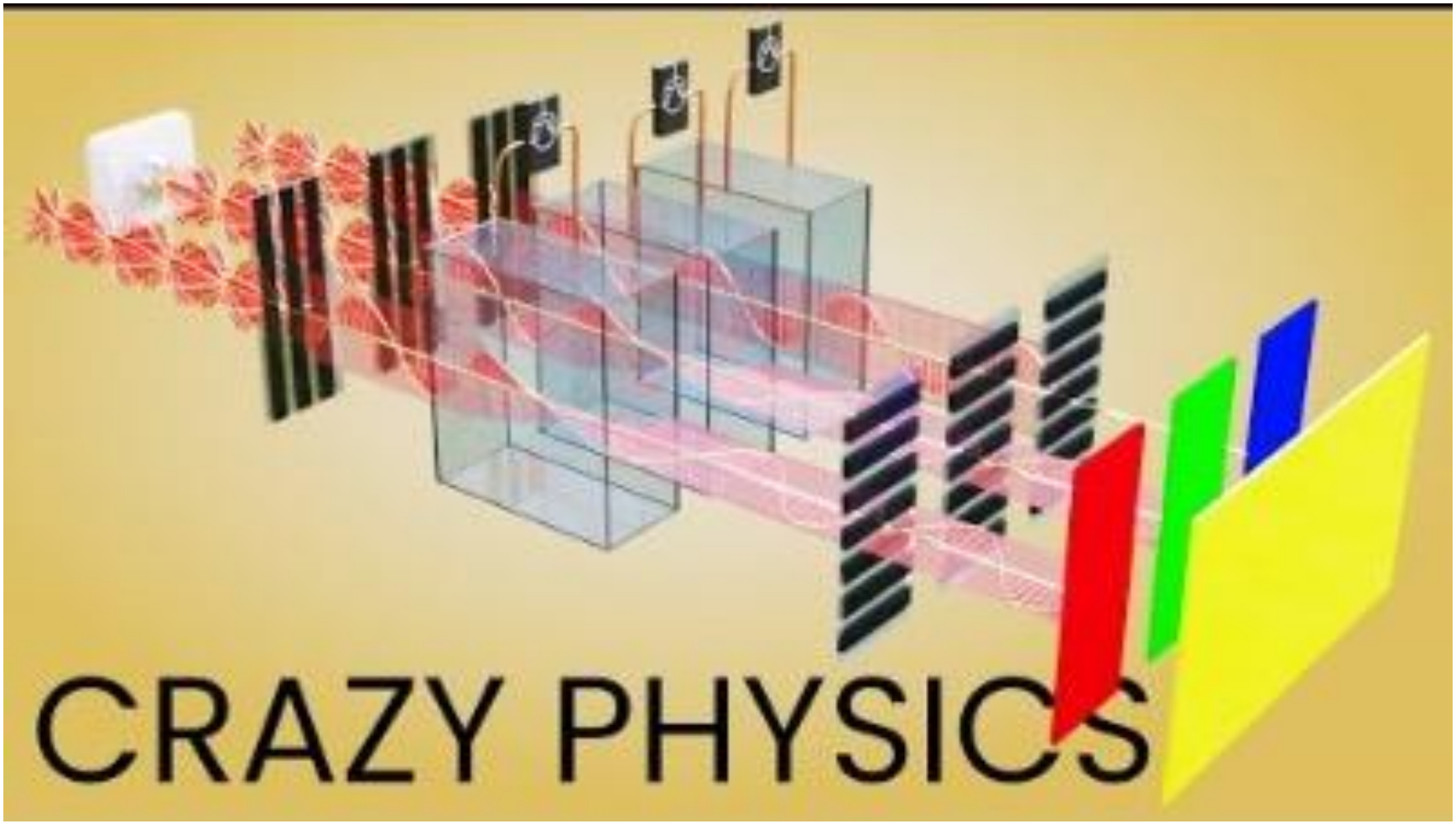
RGB images (three channel)

what we see

What we get out of the

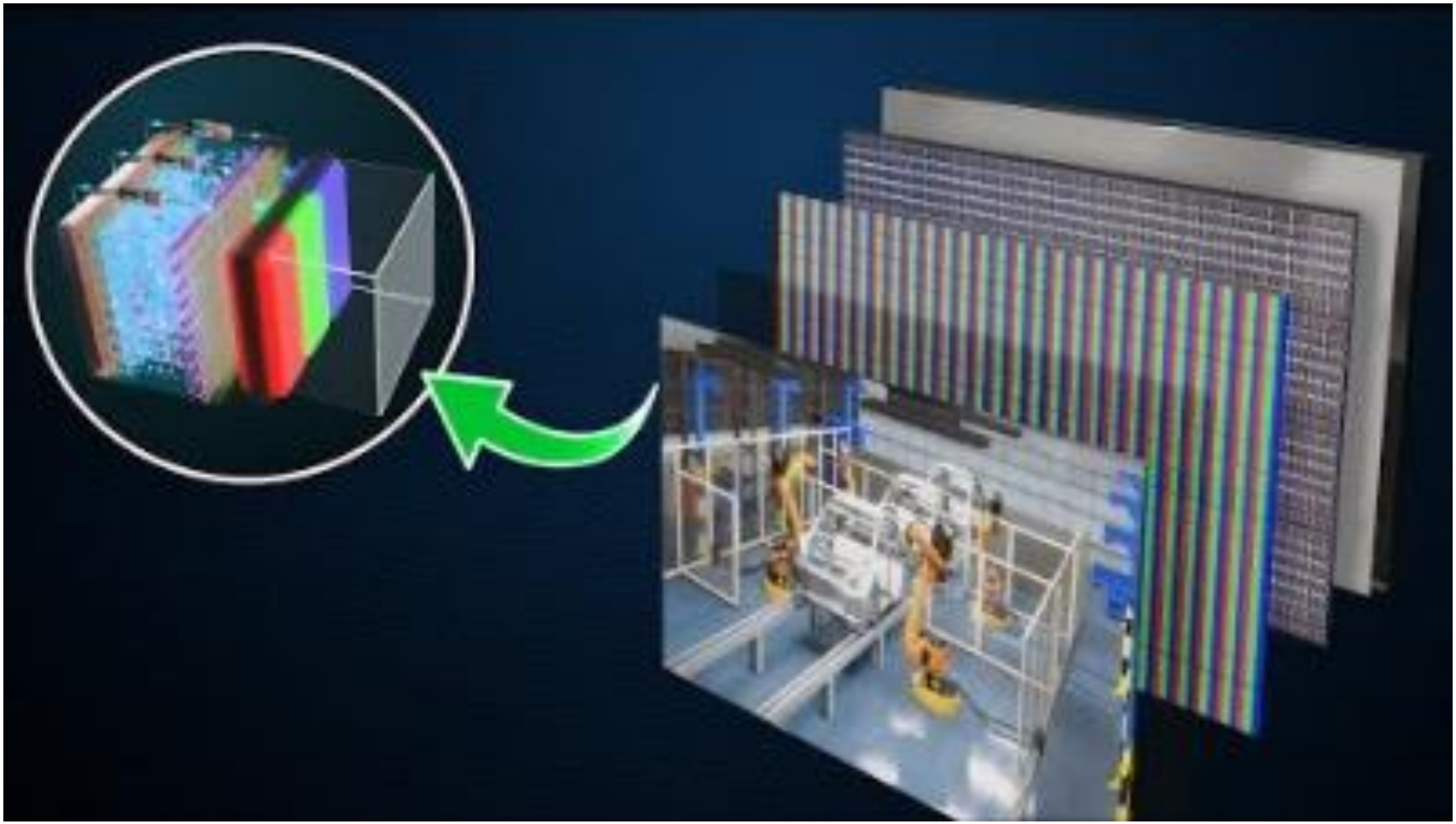
From now on: what to do with these RGB images!

How LED Display works



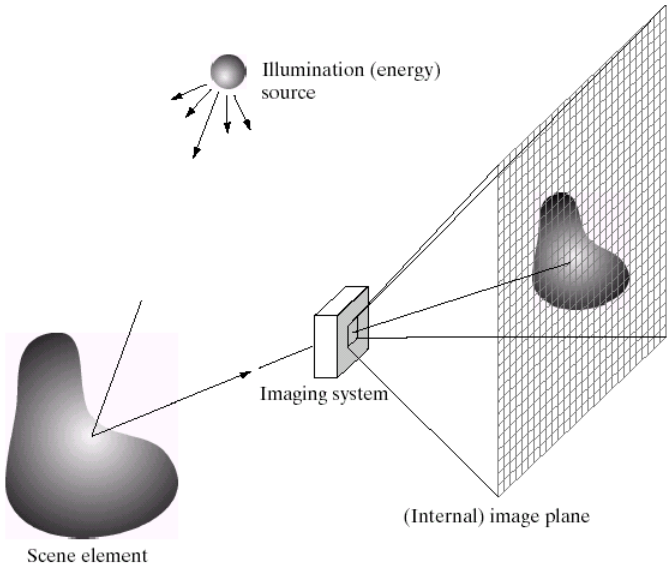
https://www.youtube.com/watch?v=96QwqOZ4xjE

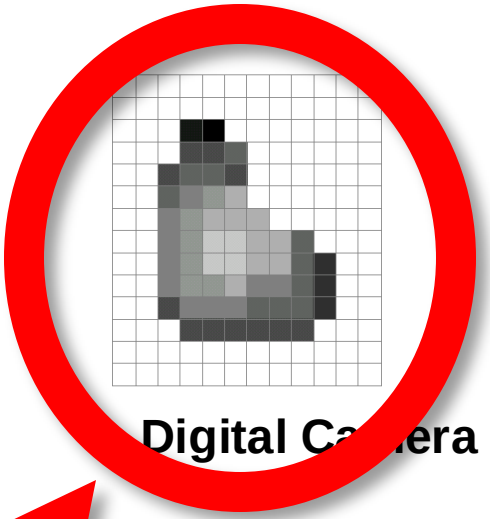
How OLED Display works

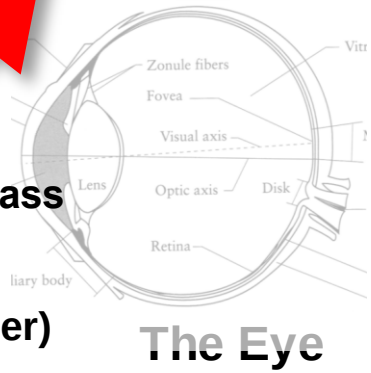


https://www.youtube.com/watch?v=xAMhX3Drq14

What is an image?







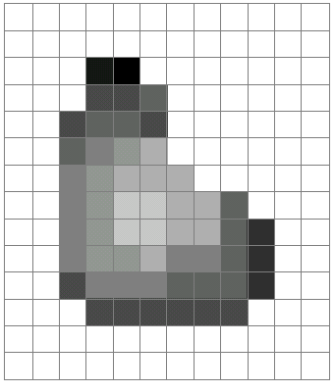
We'll focus on these in this class

(More on this process later)

Source: A. Efros

What is an image?

- A grid (matrix) of intensity values



=

255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255
255	255	127	145	145	175	127	127	95	47	255	255
255	255	74	127	127	127	95	95	95	47	255	255
255	255	255	74	74	74	74	74	74	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255

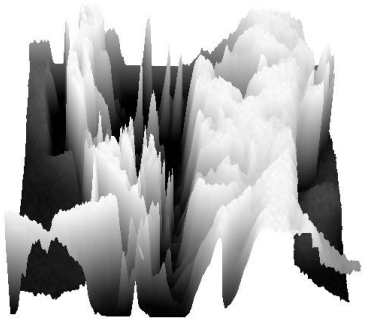
(common to use one byte per value: 0 = black, 255 = white)

What is an image?

Can think of a (grayscale) image as a **function** *f* from R^2 to R:

- $f(x,y)$ gives the **intensity** at position (x,y)

snoop



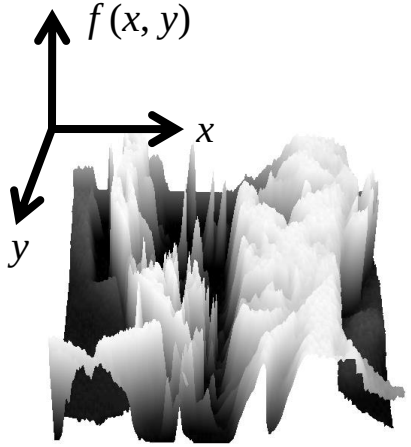
3D view

  - A **digital** image is a discrete (**sampled**, **quantized**) version of this function

What is an image?

- Can think of a (grayscale) image as a **function** f from R^2 to R:
  - $f(x,y)$ gives the **intensity** at position (x,y)

snoop

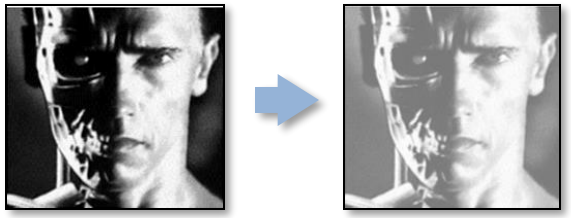


3D view

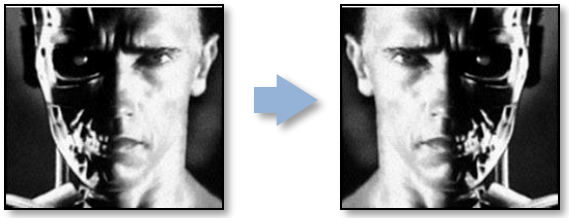
  - A **digital** image is a discrete (**sampled**, **quantized**) version of this function

Image transformations

- As with any function, we can apply operators to an image



$g(x,y) = f(x,y) + 20$

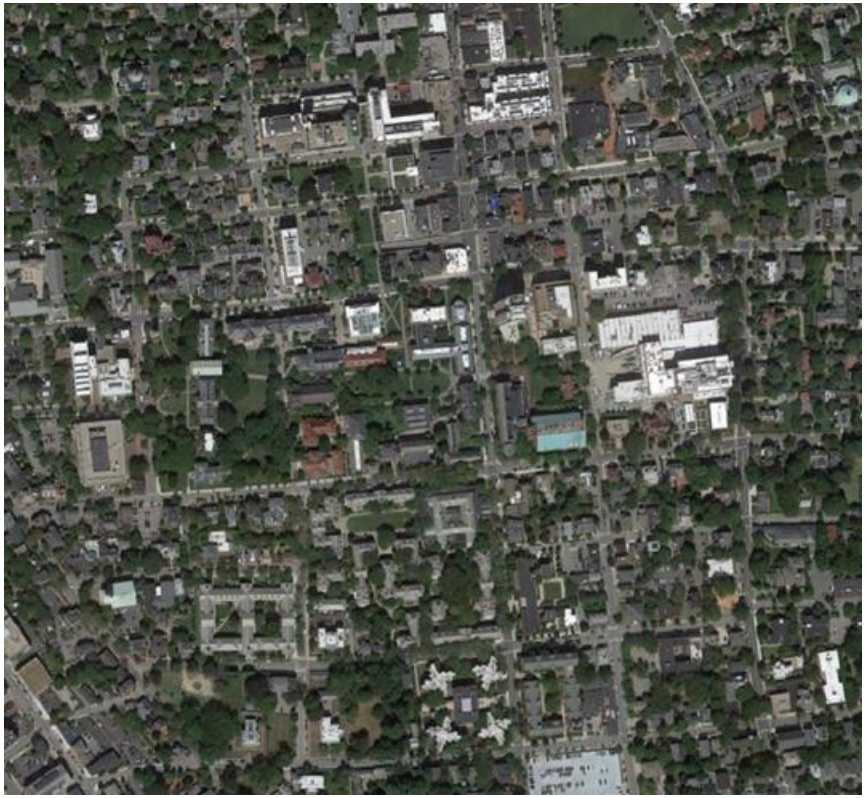


$g(x,y) = f(-x,y)$

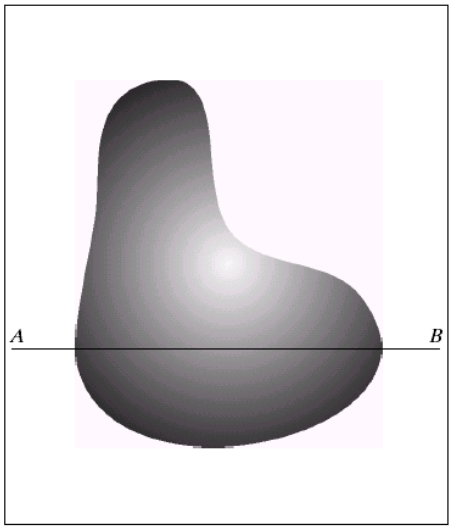
- Today we'll talk about a special kind of operator, *convolution* (linear filtering)

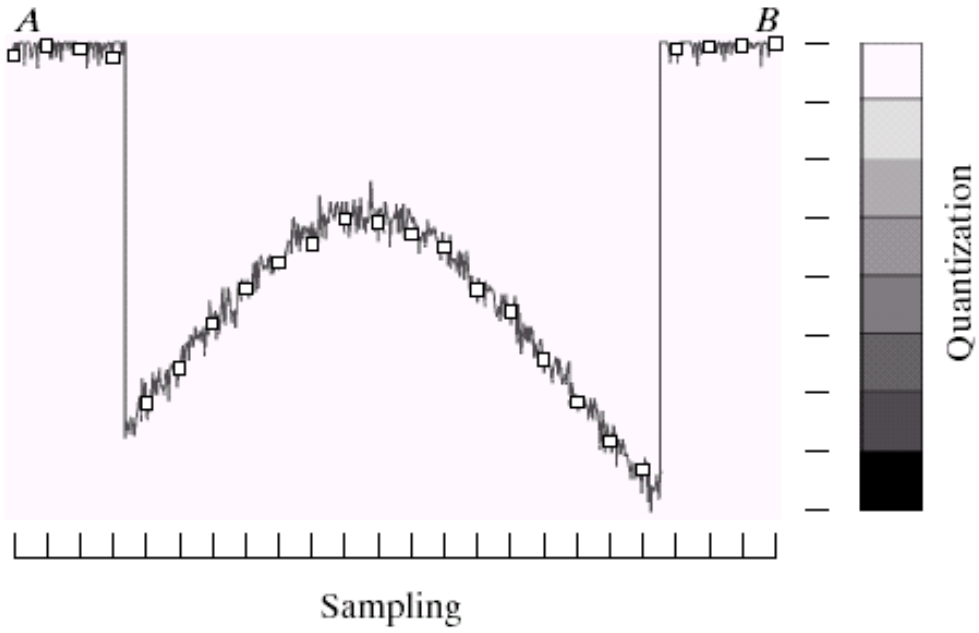
Resolution – geometric vs. spatial resolution

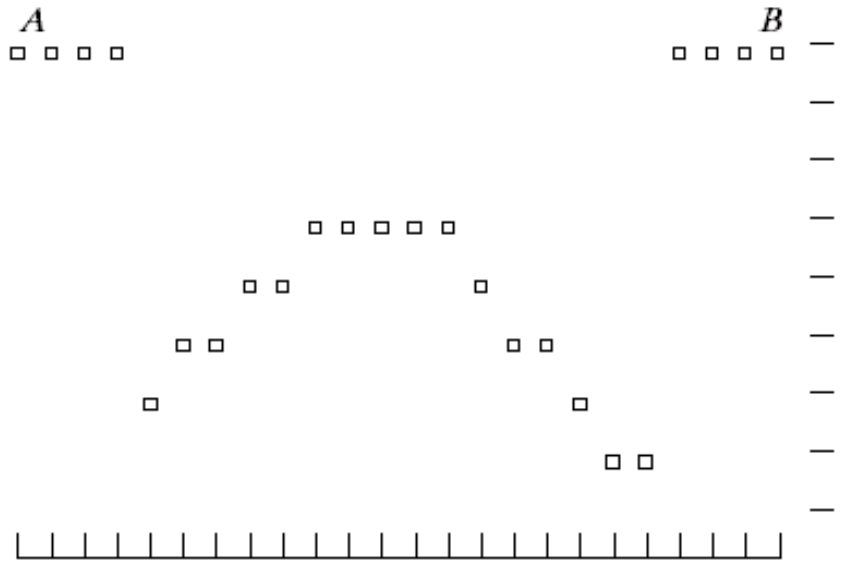
Both images are ~500x500 pixels



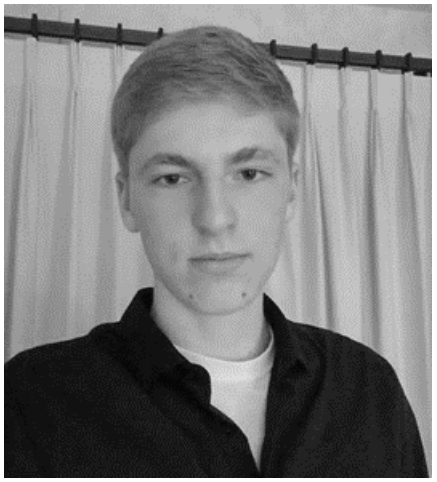
Quantization





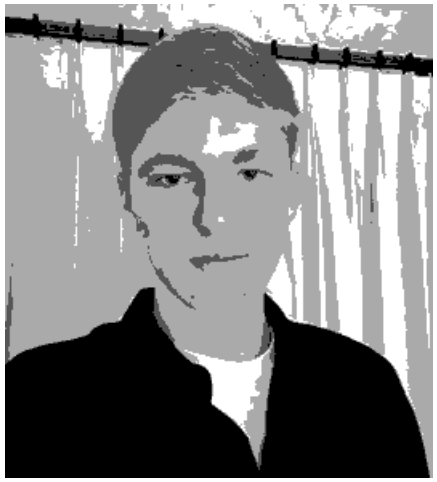


Quantization Effects – Radiometric Resolution



8 bit – 256 levels

4 bit – 16 levels



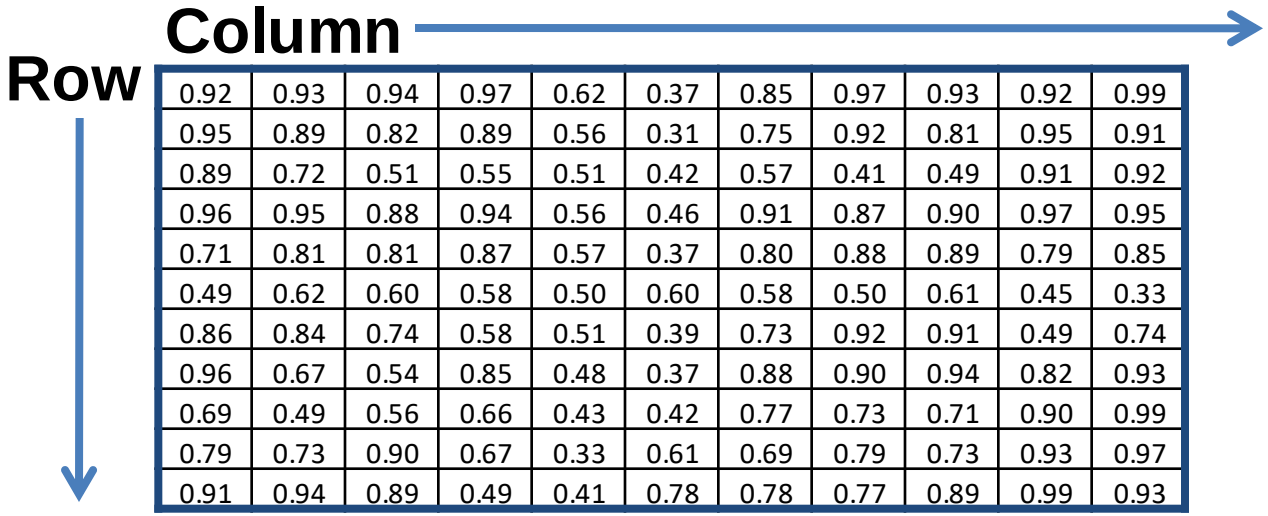
2 bit – 4 levels

1 bit – 2 levels

Images in Python Numpy

N x M grayscale image “im”

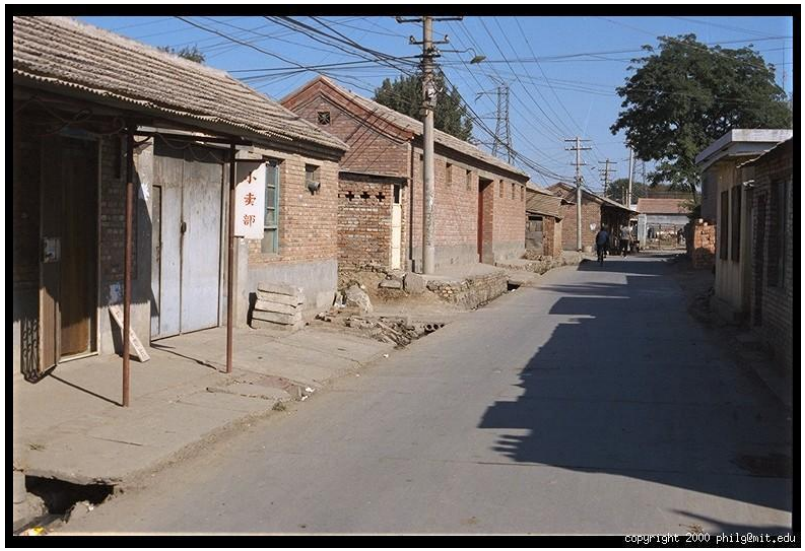
- im[0,0] = top-left pixel value
- im[y, x] = y pixels down, x pixels to right
- im[N-1, M-1] = bottom-right pixel

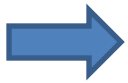


0.92	0.93	0.94	0.97	0.62	0.37	0.85	0.97	0.93	0.92	0.99
0.95	0.89	0.82	0.89	0.56	0.31	0.75	0.92	0.81	0.95	0.91
0.89	0.72	0.51	0.55	0.51	0.42	0.57	0.41	0.49	0.91	0.92
0.96	0.95	0.88	0.94	0.56	0.46	0.91	0.87	0.90	0.97	0.95
0.71	0.81	0.81	0.87	0.57	0.37	0.80	0.88	0.89	0.79	0.85
0.49	0.62	0.60	0.58	0.50	0.60	0.58	0.50	0.61	0.45	0.33
0.86	0.84	0.74	0.58	0.51	0.39	0.73	0.92	0.91	0.49	0.74
0.96	0.67	0.54	0.85	0.48	0.37	0.88	0.90	0.94	0.82	0.93
0.69	0.49	0.56	0.66	0.43	0.42	0.77	0.73	0.71	0.90	0.99
0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

Grayscale intensity

Color





R

G

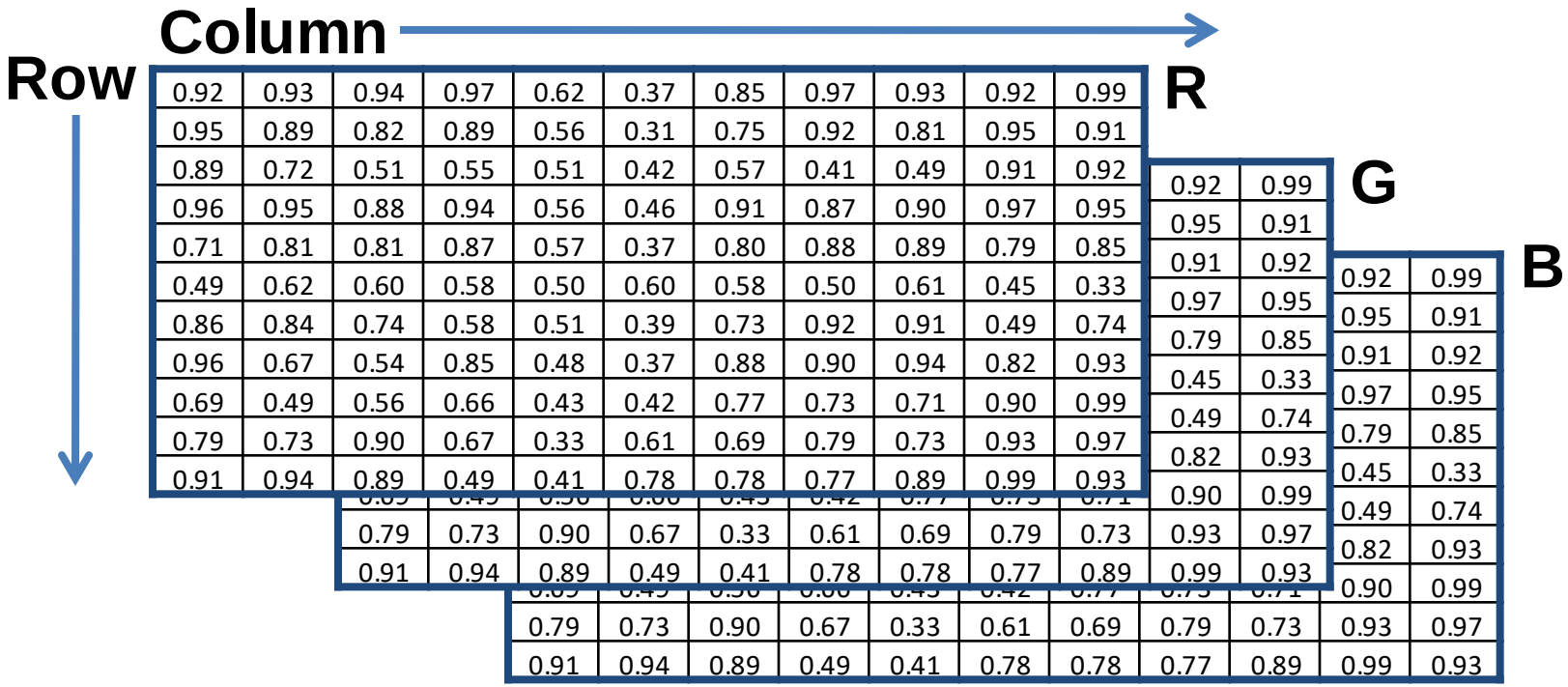
B

James Hays

Images in Python Numpy

N x M RGB image “im”

- im[0,0,0] = top-left pixel value in R-channel
- Im[x, y, b] = x pixels to right, y pixels down in the b^{th} channel
- Im[N-1, M-1, 2] = bottom-right pixel in B-channel



0.92	0.93	0.94	0.97	0.62	0.37	0.85	0.97	0.93	0.92	0.99
0.95	0.89	0.82	0.89	0.56	0.31	0.75	0.92	0.81	0.95	0.91
0.89	0.72	0.51	0.55	0.51	0.42	0.57	0.41	0.49	0.91	0.92
0.96	0.95	0.88	0.94	0.56	0.46	0.91	0.87	0.90	0.97	0.95
0.71	0.81	0.81	0.87	0.57	0.37	0.80	0.88	0.89	0.79	0.85
0.49	0.62	0.60	0.58	0.50	0.60	0.58	0.50	0.61	0.45	0.33
0.86	0.84	0.74	0.58	0.51	0.39	0.73	0.92	0.91	0.49	0.74
0.96	0.67	0.54	0.85	0.48	0.37	0.88	0.90	0.94	0.82	0.93
0.69	0.49	0.56	0.66	0.43	0.42	0.77	0.73	0.71	0.90	0.99
0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

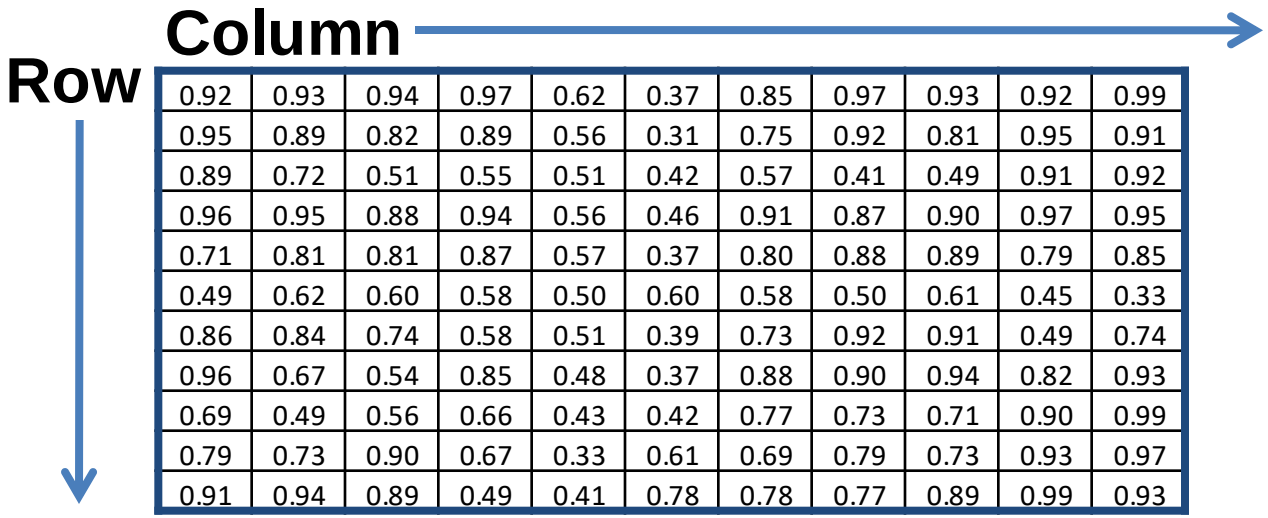
0.92	0.99									
0.95	0.91									
0.91	0.92									
0.97	0.95									
0.79	0.85									
0.45	0.33									
0.49	0.74									
0.82	0.93									
0.90	0.99									
0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

0.92	0.99									
0.95	0.91									
0.91	0.92									
0.97	0.95									
0.79	0.85									
0.45	0.33									
0.49	0.74									
0.82	0.93									
0.90	0.99									
0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

Images in Python Numpy

Take care between types!

- uint8 (values 0 to 255) – io.imread("file.jpg")
- float32 (values 0 to 255) – io.imread("file.jpg").astype(np.float32)
- float32 (values 0 to 1) – img_as_float32(io.imread("file.jpg"))



0.92	0.93	0.94	0.97	0.62	0.37	0.85	0.97	0.93	0.92	0.99
0.95	0.89	0.82	0.89	0.56	0.31	0.75	0.92	0.81	0.95	0.91
0.89	0.72	0.51	0.55	0.51	0.42	0.57	0.41	0.49	0.91	0.92
0.96	0.95	0.88	0.94	0.56	0.46	0.91	0.87	0.90	0.97	0.95
0.71	0.81	0.81	0.87	0.57	0.37	0.80	0.88	0.89	0.79	0.85
0.49	0.62	0.60	0.58	0.50	0.60	0.58	0.50	0.61	0.45	0.33
0.86	0.84	0.74	0.58	0.51	0.39	0.73	0.92	0.91	0.49	0.74
0.96	0.67	0.54	0.85	0.48	0.37	0.88	0.90	0.94	0.82	0.93
0.69	0.49	0.56	0.66	0.43	0.42	0.77	0.73	0.71	0.90	0.99
0.79	0.73	0.90	0.67	0.33	0.61	0.69	0.79	0.73	0.93	0.97
0.91	0.94	0.89	0.49	0.41	0.78	0.78	0.77	0.89	0.99	0.93

Next class: Filtering